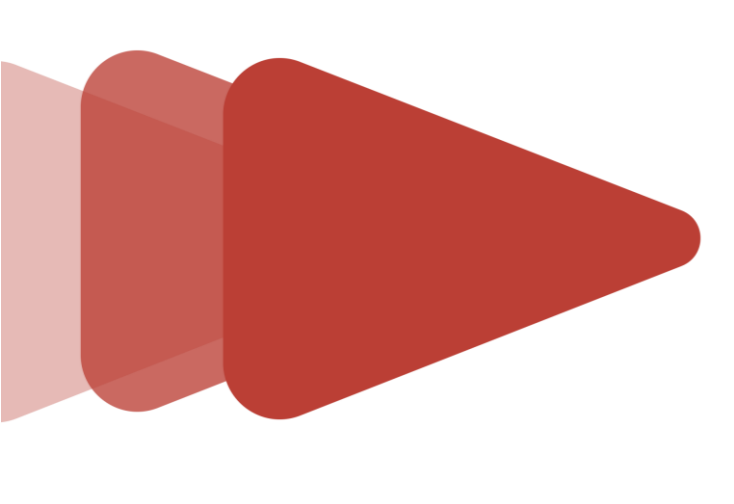




Estándar de Desarrollo Tecnológico Backend-Arquitectura Resiliente

NAC-PTITARE26-00000006

***Gestión de Datos y Arquitectura Tecnológica
Departamento de Planificación TI y Gestión de Información -Datos
Dirección Nacional de Tecnología***

2026

Versión: 1.0.0

Cláusula de Responsabilidad

Los firmantes de este documento, en representación de sus unidades administrativas a las que pertenecen, aceptan y aprueban el presente documento y a su vez se comprometen a respaldar su cumplimiento, hacer difusión del mismo en sus respectivas unidades y a sugerir mejoras para versiones futuras en el caso de que se las identifiquen.

Dirección Nacional de Tecnología		Fecha: febrero 2026	Versión: 1.0.0	PÁGINAS 55
NOMBRE Y CARGO			FIRMA	
APROBADO POR:	Ing. Paulina Correa DIRECTOR NACIONAL DE TECNOLOGÍA			
REVISADO POR:	Ing. Johanna Chagñay JEFE NACIONAL DEPARTAMENTO PLANIFICACIÓN TI Y GESTIÓN INFORMACIÓN – DATOS (E)			
	Ing. Joffre Taco JEFE NACIONAL DEPARTAMENTO DESARROLLO			
	Ing. Michel Cisneros JEFE NACIONAL DEPARTAMENTO SEGURIDAD INFORMÁTICA			
	Ing. Alexandra Pasquel COORDINADOR NACIONAL DESARROLLO Y MANTENIMIENTO			
	Ing. Edison Cayambe COORDINADOR NACIONAL GESTIÓN DE DATOS Y ARQUITECTURA TECNOLÓGICA		 Firmado electrónicamente por: EDISON ESTUARDO CAYAMBE CHICAIZA Validar únicamente con FirmaEC	
	Ing. Susana Samaniego COORDINADOR NACIONAL CONTROL Y ASEGURAMIENTO DE CALIDAD			
	Ing. Paola Alarcon EXPERTO NACIONAL DE CONTROL Y ASEGURAMIENTO DE CALIDAD		 Firmado electrónicamente por: PAOLA ALEXANDRA ALARCON LEORO Validar únicamente con FirmaEC	

	Ing. Carlos Anchundia ESPECIALISTA DE DESARROLLO	 Firmado electrónicamente por: CARLOS EDUARDO ANCHUNDIA VALENCIA Validar únicamente con FirmaEC
ELABORADO POR:	Ing. David Tamayo EXPERTO NACIONAL DE ARQUITECTURA DE SOLUCIONES TECNOLÓGICAS	 Firmado electrónicamente por: FRANKLIN DAVID TAMAYO ROMO Validar únicamente con FirmaEC
	Ing. Jorge Armijo EXPERTO NACIONAL DE ARQUITECTURA DE SOLUCIONES TECNOLÓGICAS	 Firmado electrónicamente por: JORGE LUIS ARMIJO QUITO Validar únicamente con FirmaEC

Información del Documento

Resumen:	En el presente documento se definen los estándares para la implementación de aplicaciones de microservicios basados en el Framework Quarkus.
Alcance:	Definición de lineamientos para normar el uso del Framework Quarkus, también se da lineamientos de temas como: manejo de logs y excepciones, administración de catálogos y parámetros generales.
Audiencia:	Personal de la Dirección Nacional de Tecnología.

Revisiones al Documento

Versión -Fecha	Autor	Descripción
1.0.0 / 2023-09	Christian Arreaga	Emisión Inicial. Esta versión corresponde a una versión preliminar del estándar final que se continuará editando y <u>no</u> incluye las definiciones para: • Aplicaciones de procesamiento por lotes. • Catálogos y parámetros generales • Patrones de diseño • Escalabilidad • Mensajería • Servicios SOAP • Interoperabilidad • Cache de segundo nivel • Pistas de Auditoría • Compilación nativa • Contenerización • Despliegue en ambiente basado en Kubernetes • Ejemplos o arquetipos base De igual manera la versión futura implicará el afinamiento de las reglas dispuestas en esta versión, así como también todos los documentos, definiciones, tecnologías archivos de reglas entre otros necesarios para la construcción de lógica de backend.
1.0.0 / 2025-09	Santiago Simbaña	Reglas orientadas a la correcta aplicación de Domain Driven Design (DDD)
1.0.0 / 2025-09	Jorge Armijo	Actualización documento: • Actualización general en la redacción del documento, se ha orientado con referencia a la arquitectura de referencia general de software. • Firmas de responsabilidad • Introducción, haciendo énfasis en la importancia del documento. • Mejorar la redacción del objetivo del documento.

		• Se renombraron las secciones y se generaron subsecciones para esclarecer el ámbito de las reglas definidas. • Se agregaron párrafos explicando la sección principal y el contenido de las reglas definidas. • Se agregó por cada regla planteada una utilidad para fortalecer la explicación. • Se reorganizaron las reglas del documento dentro de las secciones para una estructura coherente. • Se puso el mismo formato en los cuadros de texto y ejemplos de reglas. • Se resaltaron algunas reglas con los mismos colores para validar si es pertinente su unificación.
1.0.0 / 2026-02	David Tamayo	Revisión integral del documento, afinamiento de la estructura general y actualización de las reglas definidas, con el fin de alinearlas y asegurar su correcta aplicación conforme al framework Quarkus y a las buenas prácticas arquitectónicas institucionales.

Enmienda del Documento

El departamento de planificación TI y gestión información tiene la potestad de actualizar el contenido de los Anexos cuando lo considere necesario y no requerirá recolección de firmas para su aprobación. Sin embargo, se comunicarán a las áreas involucradas los cambios realizados, quienes podrán indicar sus observaciones al respecto en los siguientes ocho días laborables, luego de lo cual se considerarán aceptadas, y se procederá a su publicación.

La aplicación de las modificaciones al estándar en proyectos con desarrollos terminados o en curso será consensuado con el departamento de desarrollo, con el fin de evaluar el impacto (positivo o negativo) que tenga sobre el producto o sobre la calidad de este, adicionalmente se debe informar de los acuerdos a las Coordinaciones de Control y Aseguramiento de Calidad (QA) y Centro de Cómputo.

Ante cualquier sugerencia de mejora al documento debe ser comunicado al Departamento de Planificación TI y Gestión de Información - Datos para su análisis, evaluación de factibilidad y posterior actualización de la documentación en caso de considerarse necesario.

En caso de ser necesario emitir un lineamiento técnico, aclaración o alcance al presente documento, este podrá ser emitido mediante memorando suscrito por el Jefe Departamental de Planificación TI y Gestión de Información - Datos en coordinación con la Dirección Nacional de Tecnología; y se mantendrá como una disposición vigente hasta su incorporación en la siguiente actualización del documento.

Las modificaciones realizadas deben ser comunicadas al Departamento de Gestión de la Calidad, con la finalidad de llevar seguimiento de las versiones del documento.

Tabla de contenido

1.	Introducción	8
1.1.	Objetivo.....	8
2.	Estándares Generales	9
2.1.	Cumplimiento Normativo y Referencia Cruzada	9
2.2.	Versiones de Tecnologías	9
2.3.	Calidad de Código.....	10
2.4.	Aseguramiento de Calidad y Trazabilidad	10
2.5.	Catálogos y Parámetros Generales.....	11
2.6.	Duplicación de Código	11
3.	Diseño de API's	11
3.1.	Generales	12
4.	Pruebas Unitarias	12
4.1.	Desarrollo guiado por pruebas (TDD).....	12
4.2.	Implementación de pruebas unitarias	13
5.	Implementación	14
5.1.	Estructura y nomenclatura.....	14
5.2.	Compilación nativa (GraalVM).....	17
5.3.	Convenciones de API RESTFul	17
5.3.1.	Framework y versión	17
5.3.2.	Ruta base de la aplicación	17
5.3.3.	Formato de interoperabilidad.....	17
5.3.4.	Proveedores de serialización.....	18
5.3.5.	Semántica HTTP	18
5.3.6.	Códigos de estado	19
5.3.7.	Autenticación y autorización	20
5.4.	Paginación, filtrado y ordenación.....	20
5.5.	Documentación de la API	22
5.6.	Servicios de Dominio.....	22
5.7.	Servicios de Aplicación	24
5.8.	Mappers (MapStruct).....	25
5.9.	Uso de DTOs (Request/Response).....	26
5.10.	REST	27
5.11.	Configuración de URI (Uniform Resource Identifier).....	28

5.12.	Respuestas	32
6.	Seguridad	33
7.	Eventos.....	34
8.	Java Persistence API – JPA	35
8.1.	Pool de conexiones.....	36
8.2.	Procedimientos almacenados	36
8.3.	Entidades	36
8.4.	Enumeraciones de dominio	37
8.5.	Repositorios de Infraestructura	38
8.6.	Clases embeddable en entidades	38
8.7.	Mapeo de relaciones en entidades.....	39
8.8.	Repositorios de Dominio	39
8.9.	Entidades de Persistencia en Infraestructura	40
9.	Validaciones de los Java Beans.....	41
9.1.	Constraints de Bean Validation	42
9.2.	Personalización de mensajes de error de validación	42
9.3.	Validación de servicios web RESTFul	43
10.	Componentes JEE	44
10.1.	Inyección de Dependencias y Contextos (CDI)	44
10.1.1.	Scopes	44
10.1.2.	Interceptores	45
10.1.3.	Decoradores.....	45
11.	Manejo de logs y excepciones	45
11.1.	Manejo de Logs / pistas de auditoria	45
11.1.1.	Implementación de mensajes de logs.....	46
11.2.	Manejo de Excepciones	47
11.2.1.	Implementación de excepciones	47
12.	Lógica parametrizable	47
12.1.	Utilitarios Compartidos	48
13.	Glosario	48
14.	Bibliografía.....	50
15.	Anexos.....	50
15.1.	Anexo 1: Recursos Relacionados.....	50
15.2.	Anexo 2: Estándares Relacionados.....	51
15.3.	Anexo 3: Versiones de tecnologías a utilizar	51
15.4.	Anexo 4: Estructura (scaffolding).....	52

1. Introducción

El presente Estándar de Desarrollo Tecnológico establece lineamientos obligatorios para el diseño, construcción, operación y evolución de servicios backend institucionales, sobre el framework de desarrollo Quarkus, que implementa la especificación MicroProfile, orientado a la construcción de aplicaciones Java modernas optimizadas para entornos cloud, contenedores y arquitecturas de microservicios, incorporando estándares empresariales como configuración externa, tolerancia a fallos, métricas, seguridad y observabilidad. El estándar prioriza la resiliencia, seguridad, observabilidad, mantenibilidad y escalabilidad, y define criterios verificables para garantizar consistencia técnica entre equipos y proyectos.

El documento no está orientado a enseñar programación, sino a normar la manera en que deben diseñarse, configurarse. Entre los aspectos regulados se incluyen:

- Diseño de APIs.
- Validaciones y manejo de excepciones.
- Gestión de logs.
- Administración de catálogos y parámetros generales.
- Acceso a datos y transaccionalidad.
- Seguridad, autenticación y autorización centralizada.
- Pruebas unitarias.

Este estándar se alinea con la Arquitectura Tecnológica de Referencia de Software y con las normativas nacionales e internacionales vigentes:

- LOPDP (Ley Orgánica de Protección de Datos Personales).
- ISO/IEC 27001:2022 en materia de seguridad de la información.
- OWASP API Security Top 10 como guía de seguridad de servicios.
- ITIL 4 y COBIT 2019 en lo referente a gestión y gobierno de TI.

Vigencia. El estándar entra en vigor desde su aprobación y DEBE aplicarse en nuevos desarrollos y en evolutivos significativos. Cuando existan restricciones de legado (Java EE en EAP), se DEBE justificar la excepción y planificar la convergencia hacia las versiones objetivo-definidas en Anexo 3.

1.1. Objetivo

El objetivo principal del presente documento es establecer un marco normativo, verificable y homogéneo para el desarrollo de servicios backend institucionales, garantizando que las soluciones se diseñen y operen bajo los principios de resiliencia, seguridad y calidad técnica.

De manera específica el presente estándar busca:

- Normar la implementación de componentes backend institucionales en Jakarta EE / Quarkus.
- Apoyar al control de la calidad y consistencia de las aplicaciones institucionales, mediante reglas de API design, validación, manejo de errores, logs estructurados, y seguridad por defecto.
- Guiar al desarrollador sobre las herramientas a utilizarse, así como también al uso de integración continua.

Los lineamientos descritos en este estándar se organizan en tres bloques principales:

- **Estándares generales y fundamentos técnicos:** Reglas transversales que norman la estructura de proyectos, nomenclatura, uso de librerías, calidad de código y validaciones básicas, constituyendo los requisitos mínimos para iniciar un desarrollo institucional.
- **Componentes y patrones de arquitectura:** Definiciones específicas sobre el uso del framework Quarkus, patrones de diseño y buenas prácticas de resiliencia, seguridad y observabilidad, considerados el núcleo central de este estándar.
- **Aspectos complementarios de soporte:** Lineamientos asociados al manejo de logs, gestión de excepciones, pruebas unitarias e integración con servicios web, orientados a fortalecer la operación y el aseguramiento de la calidad de las soluciones.

El estándar entra en vigor desde la fecha de aprobación, y será de aplicación obligatoria en la construcción de nuevas soluciones backend. En el caso de aplicaciones existentes que operen en plataformas legadas (Java EE en JBoss EAP), se permitirá su uso únicamente bajo justificación y con un plan de convergencia hacia Jakarta EE / Quarkus, definido en la arquitectura de solución correspondiente.

El presente estándar y los documentos complementarios como guías se encuentran publicados en el repositorio institucional Alfresco en la ruta:

Direcciones Nacionales/Dirección Nacional de Desarrollo Tecnológico/Departamento de Planificación TI/Arquitectura Estándares y Guías.

El Departamento de Planificación TI y Gestión de Información Datos es el responsable de garantizar la vigencia y control de versiones de este documento, asegurando su alineación con la Arquitectura Tecnológica de Referencia y con la normativa nacional vigente.

2. Estándares Generales

Las reglas definidas en este documento son de cumplimiento obligatorio en el desarrollo de aplicativos institucionales. Para casos de excepción no descritos en este documento se deberá consultar y gestionar la aprobación al Departamento de Planificación TI y Gestión de Información-Datos **previa cualquier implementación**. Para solicitar una excepción utilizar el “Formato solicitud excepciones a estándares” descrito en el [Anexo 1: Recursos Relacionados](#).

2.1. Cumplimiento Normativo y Referencia Cruzada

Regla: Toda implementación debe cumplir, además de este estándar, con las reglas descritas en los documentos del [Anexo 2: Estándares Relacionados](#) (seguridad, base de datos, batch, codificación) según sea el caso.

Utilidad: Garantiza consistencia institucional, evita duplicación de esfuerzos y asegura que los desarrollos cumplan con las normativas técnicas, de seguridad y de calidad establecidas.

2.2. Versiones de Tecnologías

Regla: Las versiones de las tecnologías que se pueden usar para la implementación de los proyectos institucionales son las descritas en el [Anexo 3: Versiones de tecnologías a utilizar](#).

Utilidad: Permite mantener compatibilidad, soporte oficial y estabilidad en los entornos, reduciendo riesgos asociados al uso de versiones obsoletas o no certificadas.

2.3. Calidad de Código

Regla: Cualquier dependencia externa distinta al estándar de Quarkus debe ser justificada y aprobada por el Arquitecto de Soluciones Tecnológicas asignado.

Utilidad: Evita la introducción de dependencias innecesarias que compliquen el mantenimiento o generen vulnerabilidades, garantizando transparencia en el uso de librerías externas.

Regla: Se deberá priorizar el uso de librerías soportadas institucionalmente y con comunidad activa.

Utilidad: Favorece la sostenibilidad del código, facilita actualizaciones y reduce riesgos de obsolescencia.

2.4. Aseguramiento de Calidad y Trazabilidad

Regla: Excluir de las validaciones de PMD y CheckStyle las clases generadas por herramientas de terceros como, por ejemplo:

- ✓ Clases metamodelo basadas en la especificación JPA que tienen el mismo nombre de la clase administrada en la unidad de persistencia más un underscore “_” al final de su nombre, por ejemplo, la clase de persistencia Autor.java su metamodelo es Autor_.java. Del mismo modo se deben excluir de dicha validación los siguientes archivos creados en tiempo de compilación:

```
**/*_.java  
**/target**
```

Utilidad: Evita falsos positivos en los procesos de validación automática, manteniendo la trazabilidad en el código fuente escrito por los desarrolladores.

Regla: Es obligatorio el uso de PMD con las reglas personalizadas del SRI, Ver ruta en [Anexo 1: Recursos Relacionados](#).

Utilidad: PMD ayuda a detectar código ineficiente o malas prácticas de programación, fomentando estándares de calidad uniformes en todos los proyectos.

Regla: Es obligatorio la aplicación del Checkstyle definido con los lineamientos por el SRI, Ver versión en [Anexo 1: Recursos Relacionados](#).

Utilidad: Asegura que el código mantenga un estilo uniforme, mejorando su legibilidad y reduciendo el esfuerzo de mantenimiento.

Regla: Los microservicios en Quarkus deben ejecutar SpotBugs en el build y en CI/CD (mínimo en verify). El pipeline debe fallar si existen hallazgos High; los Medium deben corregirse o justificarse. Se debe generar y publicar el reporte como evidencia. Ver en [Anexo 1: Recursos Relacionados](#)

Utilidad:

Detecta defectos comunes (nulls, equals/hashCode, concurrencia) antes de producción, elevando la confiabilidad y reduciendo retrabajo e incidentes.

Regla: Todos los proyectos deben incluir controles automáticos de escaneo de vulnerabilidades en dependencias y remediar/gestionar el hallazgo encontrado.

Utilidad: Garantiza la detección temprana de riesgos de seguridad en librerías externas, reduciendo la superficie de ataque y fortaleciendo la seguridad del software.

2.5. Catálogos y Parámetros Generales

Regla: La estrategia para el registro, acceso, actualización y desactivación de parámetros debe implementarse a nivel de base de datos.

Utilidad: Centraliza el control de la parametrización, garantizando coherencia y evitando dispersión en la gestión de configuraciones.

Regla: Todo catálogo debe estar centralizado en el módulo de ADM (Administración). Para acceder a dicha información se deben utilizar servicios genéricos ofrecidos por este módulo.

Utilidad: Facilita la reutilización, evita duplicidad y asegura la consistencia de los datos compartidos entre aplicaciones.

Regla: Para el ambiente de Contingencia se utilizarán los parámetros correspondientes a este ambiente, configurados en la infraestructura tecnológica.

Utilidad: Permite continuidad operativa en escenarios de contingencia, asegurando que las aplicaciones funcionen con los valores adecuados a cada entorno.

Regla: Los registros de los parámetros se deben cargar en cache por demanda, evitando sobrecargas en la base de datos.

Utilidad: Mejora el rendimiento de los aplicativos reduciendo la latencia y el consumo innecesario de recursos de la base de datos.

Regla: Toda parametrización sensible (usuarios, claves, tokens) debe gestionarse mediante mecanismos seguros de configuración externa (ej. Vault, ConfigMaps, Secrets).

Utilidad: Refuerza la seguridad al evitar exposición de credenciales sensibles en el código fuente o en configuraciones no protegidas.

2.6. Duplicación de Código

Regla: Se deben excluir de SonarQube las clases estructurales repetitivas ubicadas en aplicación (Request/Response), en infraestructura (Entity) y dominio, para evitar falsos positivos de duplicación.

```
**/application/**/*Request.java  
**/ application /**/*Response.java  
**/ infrastructure/**/*Entity.java  
**/domain/**/**.java
```

Utilidad: Evita que SonarQube reporte duplicación por código (DTOs y entidades) y mantiene el análisis enfocado en duplicación real de lógica.

3. Diseño de API's

La definición y construcción de APIs constituye un pilar fundamental en la arquitectura backend institucional, ya que representan el medio de interacción entre los sistemas internos, los servicios externos y los consumidores finales. Un diseño correcto de APIs garantiza consistencia, escalabilidad, seguridad y facilidad de integración, reduciendo errores de implementación y asegurando interoperabilidad entre componentes heterogéneos.

En esta sección se establece los lineamientos que deben seguirse en el diseño de APIs institucionales, incluyendo la definición de arquitectura en capas, uso correcto de métodos HTTP, aplicación de códigos de estado estándar, convenciones para URLs y versionamiento, manejo de errores estructurados, implementación de controles de seguridad, así como el soporte a paginación, filtrado, ordenación, documentación y limitación de peticiones.

En conjunto, estas reglas buscan que todas las APIs institucionales sean predecibles, estandarizadas y seguras, alineándose con buenas prácticas internacionales y con la Arquitectura Tecnológica de Referencia de Software de la institución.

3.1. Generales

Regla: Se debe utilizar la especificación OpenAPI para el diseño de los servicios RESTful. La versión de esta se puede revisar en [Anexo 3: Versiones de tecnologías a utilizar](#)

Utilidad: OpenAPI permite estandarizar la descripción de las APIs, facilita la generación de clientes automáticos, mejora la interoperabilidad y reduce errores de integración.

4. Pruebas Unitarias

Las pruebas unitarias son un componente esencial dentro del ciclo de vida del software, ya que permiten validar el correcto funcionamiento de las unidades de código (métodos, clases, servicios) de manera aislada. Su aplicación sistemática incrementa la calidad del producto, reduce defectos en fases posteriores y facilita la evolución del sistema.

En el contexto institucional, las pruebas unitarias se integran al flujo de Desarrollo Guiado por Pruebas (TDD) y a los pipelines de Integración y Entrega Continua (CI/CD), constituyendo un criterio obligatorio de aseguramiento de calidad.

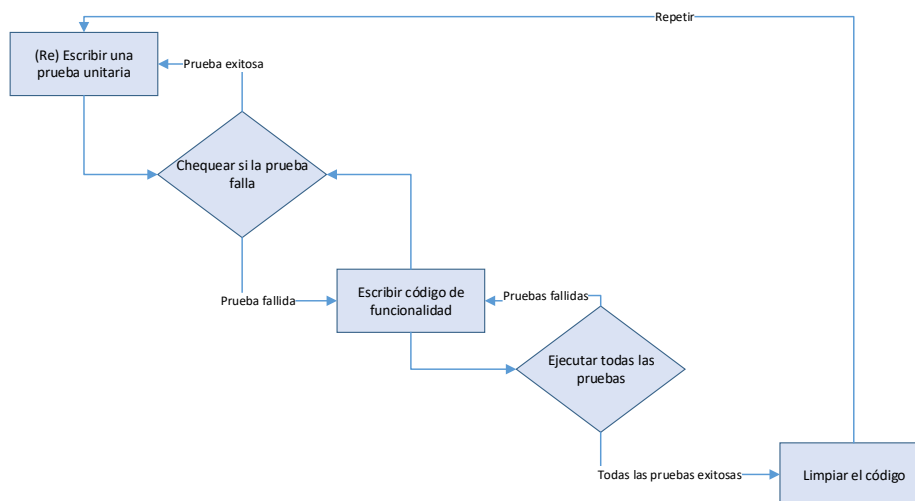
Este capítulo define los lineamientos para la implementación de pruebas unitarias en proyectos backend, especificando herramientas permitidas, cobertura mínima, prácticas recomendadas y criterios de aceptación.

4.1. Desarrollo guiado por pruebas (TDD)

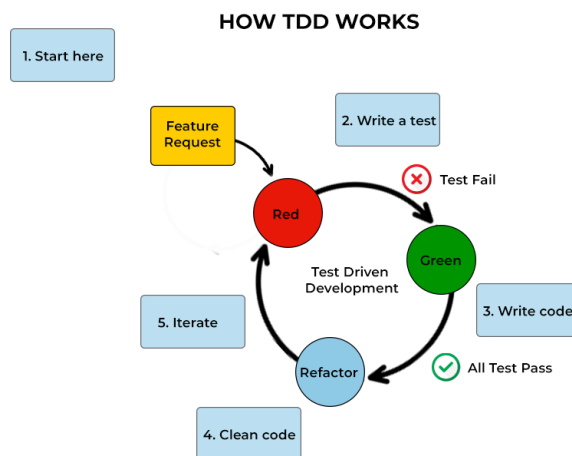
Regla: Para la implementación de un nuevo proyecto se debe aplicar la técnica de desarrollo guiado por pruebas, en inglés Test Driven Development (TDD).

Utilidad: Con el desarrollo guiado por pruebas se favorece la construcción de aplicaciones de mayor calidad; se enfoca a las necesidades del cliente facilitando el entendimiento del problema, y disminuyendo el tiempo de depuración de errores.

La siguiente imagen muestra el ciclo de desarrollo de pruebas unitarias con TDD:



spiceworks



Ciclo de desarrollo de TDD

4.2. Implementación de pruebas unitarias

Regla: Las pruebas unitarias DEBEN implementarse con JUnit para las validaciones y afirmaciones.

Utilidad: Garantiza un marco de aserciones uniforme en todos los proyectos.

Regla: Para pruebas unitarias de servicios de aplicación y recursos RESTful en Quarkus, las dependencias hacia infraestructura (repositorios, clientes HTTP/mensajería, persistencia, seguridad, etc.) deben aislarse mediante mocks, usando Mockito como framework estándar (por ejemplo, con @InjectMock cuando aplique).

Utilidad: Permite validar la lógica y contratos de la capa bajo prueba simulando colaboraciones y verificando interacciones, evitando acoplamiento a infraestructura, mejorando la velocidad, confiabilidad y determinismo de las pruebas.

Regla: Es obligatorio cubrir con pruebas unitarias las funcionalidades críticas o complejas; las sencillas o no críticas son opcionales, siempre que se cumpla el porcentaje de cobertura establecido.

Utilidad: Prioriza el riesgo funcional sin descuidar la métrica global de calidad.

Regla: No se requieren pruebas unitarias explícitas para: entidades de persistencia, JavaBeans (getters/setters), POJOs TO, DAOs, clases de excepciones, definiciones de interfaces, enumeraciones, static data model y clases anotadas con @Produces, siempre que se cumpla la cobertura definida.

Utilidad: Enfoca el esfuerzo de prueba donde existe lógica significativa.

Regla: El porcentaje de cobertura del código DEBE cumplir lo definido en el Procedimiento de Desarrollo y Mantenimiento de Aplicativos Informáticos Institucionales.

Utilidad: Asegura un umbral mínimo institucional verificable.

Regla: Todas las afirmaciones (asserts) DEBEN incluir un mensaje descriptivo.

Utilidad: Facilita el diagnóstico ante fallos de prueba.

Regla: Las pruebas unitarias DEBEN ser independientes entre sí y determinísticas.

Utilidad: Evita fallos intermitentes y permite su ejecución en cualquier orden.

Regla: En las pruebas unitarias NO DEBE registrarse información en archivos de log ni usarse el log como criterio de éxito/fallo.

Utilidad: Elimina efectos colaterales de E/S, mantiene las pruebas rápidas y reproducibles.

5. Implementación

La etapa de implementación define la estructura, organización y buenas prácticas para el desarrollo de proyectos backend institucionales. Un diseño consistente en la estructura de carpetas, nomenclatura de clases, configuración de servicios, definición de URIs, validaciones y respuestas es fundamental para garantizar mantenibilidad, escalabilidad y alineación con la Arquitectura Tecnológica de Referencia de Software.

Este capítulo establece los lineamientos que deben seguirse para estructurar proyectos, configurar servicios RESTful, definir URIs y estandarizar respuestas y validaciones, asegurando consistencia técnica en todos los desarrollos institucionales.

5.1. Estructura y nomenclatura

Regla: El nombrado de los proyectos de microservicios basados en Quarkus debe seguir la siguiente estructura “sri-[solucion]-[bounded-context]-[capacidad]”, todo en minúsculas.

Donde:

- **[solucion]:** identifica la solución (p. ej., anexos, comprobantes).
- **[bounded-context]:** contexto delimitado (DDD) (p. ej., parametrizacion, catalogos, seguridad).
- **[capacidad]:** tipo de capacidad expuesta (p. ej., api, worker, scheduler, ingestor, notifier, adapter).

Utilidad: Establece un estándar de identificación simple y predecible que facilita la trazabilidad y la gestión de proyectos.

Ejemplo: sri-comprobantes-recepcion-api

Regla: El paquete base del código DEBE ubicarse bajo `ec.gob.sri.[solucion].[bounded-context].[capacidad]` y las clases DEBEN nombrarse en `UpperCamelCase`.

Donde:

- **[solucion]**: identifica la solución (p. ej., anexos, comprobantes).
- **[bounded-context]**: contexto delimitado (DDD) (p. ej., parametrización, catálogos, seguridad).
- **[capacidad]**: tipo de capacidad expuesta (p. ej., api, worker, scheduler, ingestor, notifier, adapter).

Utilidad: Mantiene consistencia y reduce la curva de aprendizaje.

Regla: La estructura (scaffolding) para los proyectos que se debe seguir es la siguiente:

```
src/
└─ main/
   └─ java/
      └─ ec.gob.sri/
         └─ [solucion]/
            └─ [bounded-context]/
               └─ [capacidad]/
                  └─ domain/
                     └─ model/
                        └─ entidad/ -> Entidades del dominio (sin anotaciones JPA)
                        └─ vo/      -> Value Objects inmutables
                        └─ enums/   -> Enumeraciones del dominio
                     └─ repository/ -> Interfaces de acceso al repositorio (puertos)
                     └─ service/   -> Reglas de negocio transversales (servicios del dominio)
                  └─ application/
                     └─ dto/      -> Objetos de entrada/salida (DTOs)
                     └─ service/  -> Casos de uso (orquestadores)
                  └─ infrastructure/
                     └─ rest/     -> Controladores REST (exposición de API)
                     └─ persistence/
                        └─ entity/ -> Entidades JPA/Panache
                        └─ mapper/  -> Conversión entre entidades JPA y dominio
                        └─ repository/-> Implementación de interfaces del dominio
                  └─ shared/
                     └─ exception/ -> Excepciones reutilizables
                     └─ response/  -> Estructuras de respuesta común
                     └─ utils/     -> Utilidades generales
```

Utilidad: Estandariza la organización de proyectos, simplifica la curva de aprendizaje y facilita auditorías técnicas. (Detalle revisar en el [Anexo 4: Estructura \(scaffolding\)](#)).

Regla: La estructura de nombrado de las clases según sea el tipo se debe regir conforme la siguiente tabla y definiciones:

- ✓ El nombrado de las clases debe ser en notación UpperCamelCase.
- ✓ El paquete principal debe ser **ec.gob.sri.[solucion].[bounded-context].[capacidad]**.

Utilidad: Mantener un estándar de nombrado

Ejemplo: Revisar el Anexo 1: Recursos Relacionados, la sección de proyecto base.

```
src
├── main
│   ├── docker
│   └── java\ec\gob\sri\api\catalogo
│       ├── application
│       │   ├── dto
│       │   │   ├── ErrorResponse.java
│       │   │   ├── ParametroAmbienteRequest.java
│       │   │   └── ParametroAmbienteResponse.java
│       │   ├── service
│       │   │   └── GestionarParametroAmbiente.java
│       │   ├── domain
│       │   │   ├── model
│       │   │   │   └── entity
│       │   │   │       └── ParametroAmbiente.java
│       │   │   ├── enums
│       │   │   │   └── Ambiente.java
│       │   │   ├── repository
│       │   │   │   └── ParametroAmbienteRepository.java
│       │   │   └── infrastructure
│       │   │       ├── filter
│       │   │       │   └── XssProtectionFilter.java
│       │   │       ├── persistence
│       │   │       │   ├── entity
│       │   │       │   │   └── ParametroAmbienteEntity.java
│       │   │       │   ├── mapper
│       │   │       │   │   └── ParametroAmbienteMapper.java
│       │   │       │   └── repository
│       │   │       │       ├── ParametroAmbientePanacheRepository.java
│       │   │       │       └── ParametroAmbienteRepositoryImpl.java
│       │   │       └── rest
│       │   │           └── ParametroAmbienteResource.java
│       │   └── shared\exception
│       │       └── ManejadorExcepcion.java
```

5.2. Compilación nativa (GraalVM)

Regla: El uso de GraalVM para compilación nativa en microservicios Quarkus es opcional y debe aplicarse únicamente cuando exista una justificación técnica (por ejemplo: requerimientos de tiempo de arranque, consumo de memoria, densidad de pods, o escalado rápido). Para proyectos que adopten compilación nativa, se debe asegurar compatibilidad de dependencias con GraalVM.

Utilidad: La compilación nativa puede reducir significativamente el tiempo de arranque y el consumo de memoria, mejorando el escalado y la eficiencia en entornos cloud y Kubernetes. La restricción por justificación evita complejidad innecesaria y problemas de compatibilidad en dependencias y reflexión.

Regla: Para microservicios Quarkus que se ejecuten como **procesos efímeros** (por ejemplo: *workers*, *jobs* programados, *consumidores de eventos* de corta duración o tareas bajo demanda) se **DEBE** priorizar la compilación nativa con **GraalVM**, debido a que estos componentes se crean y terminan con frecuencia.

Utilidad: En procesos de vida corta, el tiempo de arranque representa una parte significativa del tiempo total de ejecución. La compilación nativa reduce el *startup time* y el consumo de memoria, mejorando la eficiencia del clúster y permitiendo ejecuciones más rápidas y escalables en Kubernetes (especialmente cuando hay picos de ejecución o múltiples instancias simultáneas).

5.3. Convenciones de API RESTful

5.3.1. Framework y versión

Regla: La implementación de servicios RESTful debe realizarse con JAX-RS (Ver versión en [Anexo 3: Versiones de tecnologías a utilizar](#)).

Utilidad: Estandariza y optimiza la exposición de APIs RESTful en Quarkus para facilitar mantenimiento y asegurar buen rendimiento.

5.3.2. Ruta base de la aplicación

Regla: Todos los endpoints RESTful deben implementarse en la carpeta `rest`, ubicada en: `ec.gob.sri/[solucion]/[bounded-context]/[capacidad]/infrastructure/rest`. Además, cada microservicio debe configurar el prefijo de la ruta base de la API en el archivo `src/main/resources/application.properties`, mediante el parámetro `quarkus.http.root-path`.

Utilidad: Centraliza y estandariza la ubicación de los endpoints y su ruta base, facilitando el mantenimiento y el consumo de las APIs.

Ejemplo: `quarkus.http.root-path=/catalogo/api/v1`

5.3.3. Formato de interoperabilidad

Regla: El formato por defecto para el intercambio de información a través de los servicios web debe ser JSON.

Utilidad: JSON es un estándar ampliamente adoptado, ligero y fácil de integrar, lo que asegura la interoperabilidad entre sistemas y plataformas heterogéneas.

5.3.4. Proveedores de serialización

Regla: Para implementaciones de serialización, deserialización y transformación de datos se debe utilizar un framework estándar soportado por la plataforma (JSON-B o Jackson). El uso de Jackson se considerará cuando exista requerimiento funcional o técnico que lo justifique (por ejemplo, serializadores avanzados, compatibilidad con terceros).

Utilidad: Proporciona soporte robusto y probado para formatos JSON y XML, ampliamente compatible con Quarkus. Jackson es una biblioteca de Java de usos múltiples para procesar formato de datos JSON y XML disponible para la plataforma StAX, JAXB, etc. Se integra con algunas implementaciones de JAX-RS (Jersey, Apache CXF, RESTEasy, Restlet) y el marco de trabajo de Spring.

5.3.5. Semántica HTTP

Regla: Utilizar correctamente los métodos HTTP (GET, POST, PUT, DELETE, PATCH) de acuerdo con su propósito específico y semántica.

Utilidad: Garantiza que las acciones sobre los recursos sean claras y semánticamente correctas, asegurando que la API sea intuitiva, fácil de integrar y seguir las convenciones estándar de la web.

Ejemplo:

Método HTTP	Idempotente	Lineamiento
GET	Si	Utilizar para capacidades de lectura y obtención de datos. Este método no puede cambiar datos. No exponga operaciones inseguras a través de este método.
PUT	Si	Utilizar para capacidades de actualización.
PATCH	No	Utilizar para capacidades de actualización parcial de un recurso existente. Se debe aplicar cuando solo se requiera modificar ciertos atributos sin reemplazar completamente el recurso.
DELETE	Si	Utilizar para capacidades de eliminación. Un borrado lógico, aunque representa una actualización de estados deberá desarrollarse con el verbo: "DELETE" ya que realiza una eliminación de recursos identificado por un URI.
POST	No	Utilizar para capacidades que crean nuevos recursos.
HEAD	Si	Utilizar este método para lectura y obtención de metadatos de los encabezados de respuesta. No exponga operaciones inseguras a través de este método.
OPTION	Si	Utilizar este método para comprobar la funcionalidad de un servidor web mediante petición en lugar de un recurso específico. No exponga operaciones inseguras a través de este método.
TRACE	Si	Utilizar este método para fines de depuración y diagnóstico. No exponga operaciones inseguras a través de este método.

5.3.6. Códigos de estado

Regla: Utilizar códigos de estado HTTP adecuados en las respuestas para reflejar el resultado de la operación de manera clara. Estos códigos de estado de respuesta HTTP indican si se ha completado una solicitud HTTP específica. Las respuestas se agrupan en las siguientes clases¹:

1. Respuestas informativas (100–199),
2. Respuestas satisfactorias (200–299),
3. Redirecciones (300–399),
4. Errores de los clientes (400–499),
5. y errores de los servidores (500–599).

Utilidad: Proporciona respuestas claras y facilita la integración y depuración de la API, mejorando la experiencia del desarrollador.

Ejemplo:

Código de Estado	Descripción	Contexto de Implementación	VERBO
200	OK	Utilizar para emitir un resultado de éxito general con contenido en el cuerpo de la respuesta.	GET, PATCH, POST
201	Creado	Utilizar para emitir un resultado de éxito por una creación exitosa.	POST
204	OK Sin Contenido	Utilizar para emitir un resultado de éxito general sin contenido en el cuerpo de la respuesta.	DELETE
400	Petición Incorrecta	Utilizar para emitir un resultado de error por validaciones o restricciones en los parámetros de la petición.	GET, POST, PUT, PATCH, DELETE
401	No Autorizado	Utilizar para emitir un resultado de error de la petición por que el sujeto no está autenticado y el recurso es privado.	GET, POST, PUT, PATCH, DELETE
403	Prohibido	Utilizar para emitir un resultado de error de la petición por que el sujeto no está autorizado para realizar la operación solicitada	POST, PUT, PATCH, DELETE
404	No Encontrado	Utilizar para emitir un resultado de error de la petición por ser un recurso no establecido o inexistente	GET, POST, PUT, PATCH, DELETE
500	Error Interno del Servidor	Utilizar para emitir un resultado de error general cuando el servidor lanza una excepción no controlada.	-
503	Servicio Indisponible	Utilizar para emitir un resultado de error cuando el servicio está congestionado o por tareas de mantenimiento	-

¹ Véase <https://datatracker.ietf.org/doc/html/rfc7231#section-6.5.1>

	catastros/api/v1/usuarios?sort=nombre_desc
--	--

Regla: La paginación debe incluir los siguientes campos:

- **total_resultados:** un valor entero que indique el número total de registros en la colección.
- **total_paginas:** un valor entero que indique el número total de páginas de la colección, basado en el tamaño de página actual.
- **enlaces de paginación,** que deben incluir:
 - primera: URL de la primera página de recursos.
 - ultima: URL de la última página de recursos.
 - anterior: URL de la página anterior de recursos.
 - siguiente: URL de la siguiente página de recursos.

Utilidad: La paginación es fundamental cuando se manejan grandes cantidades de datos en una API, ya que evita la sobrecarga de enviar todos los datos a la vez. Incluir estos campos en las respuestas de paginación proporciona una manera estructurada y eficiente de navegar a través de las páginas de los recursos.

Ejemplo:

```
{
  "paginacion": {
    "total_resultados": 20,
    "total_paginas": 2,
    "primera": {
      "href": " http://www.sri.gob.ec/sri-
catastros/api/v1/usuarios?order_by=-created_at&page=1&per_page=10"
    },
    "ultima": {
      "href": " http://www.sri.gob.ec/sri-
catastros/api/v1/usuarios?order_by=-created_at&page=2&per_page=10"
    },
    "siguiente": {
      "href": " http://www.sri.gob.ec/sri-
catastros/api/v1/usuarios?order_by=-created_at&page=2&per_page=10"
    },
    "anterior": null
  }
}
```

Regla: Una colección DEBE contener:

- Un campo *recursos*, el cual es una matriz que contiene varios recursos.
- Un campo paginación campo que contenga un objeto de paginación.

Utilidad: Una colección es una lista de varios recursos. Una colección se representa como un objeto JSON.

Ejemplo:

```
{
  "paginacion": {
    "total_resultados": 2,
    "total_paginas": 1,
    "primera": {
      "href": "http://www.sri.gob.ec/sri-
catastros/api/v1/sujetos?page=1&per_page=10"
    },
    "ultima": {
      "href": "http://www.sri.gob.ec/sri-
catastros/api/v1/sujetos?page=1&per_page=10"
    },
  },
}
```

```

        "siguiente": null,
        "anterior": null
    },
    "recursos": [
        {
            .....
        },
        {
            .....
        }
    ]
}

```

5.5. Documentación de la API

Regla: La API debe estar documentada utilizando herramientas como Swagger/OpenAPI, de modo que todos los endpoints, parámetros, respuestas y ejemplos sean fácilmente accesibles y comprensibles para los desarrolladores.

Utilidad: Mejora la integración con otros sistemas y facilita la comprensión de la API, permitiendo que los desarrolladores consuman y mantengan la API de manera más eficiente.

Ejemplo:

```

o Documentación en Swagger para el recurso /usuarios:
paths:
  /usuarios:
    get:
      description: "Obtiene la lista de usuario"
      responses:
        200:
          description: "Lista de usuarios"
          content:
            application/json:
              schema:
                type: array
                items:
                  $ref: '#/components/schemas/Usuario

```

5.6. Servicios de Dominio

Regla: Definir servicios de dominio cuando las reglas de negocio que involucran múltiples entidades.

Utilidad: Encapsulan reglas que involucran múltiples entidades.

Ejemplo:

```

public class TransferenciaStockService {
    public void transferir(...) { ... }
}

```

Regla: Usar domain.service para lógica de negocio transversal

Utilidad: Permite encapsular reglas de negocio que involucran múltiples entidades, evitando que una sola entidad asuma responsabilidades que no le pertenecen. Mejora la separación de responsabilidades.

Ejemplo:

```

public class ValidadorDeStock {
    public boolean validarTransferencia(Ubicacion origen, Ubicacion destino, int cantidad) {

```

```

        return origen.getStock().esSuficiente(cantidad) && destino.esActiva();
    }
}

```

Regla: El Servicio de Dominio NO debe contener lógica técnica

Utilidad: Mantiene al dominio independiente del framework o infraestructura. No debe usar JPA, Panache, ni ninguna librería de infraestructura.

Ejemplo:

```

Incorrecto:
// No usar inyección en domain.service
@Inject PersonaRepository repo;
Correcto:
// El ApplicationService se encarga de llamar este servicio, pasando los objetos necesarios

```

Regla: El Servicio de Dominio encapsula reglas que no encajan en una sola entidad

Utilidad: Cuando no hay una entidad claramente responsable de una regla de negocio, el dominio debe mantener esta lógica separada en un domain.service.

Ejemplo:

```

public class GeneradorDeReporte {
    public ReporteGenerar(List< Persona> personas, Date desde, Date hasta) {
        // lógica de negocio transversal
    }
}

```

Regla: El Servicio de Dominio no accede directamente a la base de datos

Utilidad: No se debe acoplar a persistencia. Si necesita datos, estos deben ser proporcionados por la capa application.

Ejemplo:

Correcto:

```

public class EvaluadorDeEstado {
    public boolean puedeActivarse(Persona persona) {
        return persona.totalDeuda() > 0;
    }
}

```

Incorrecto:

```

personaEntity.findById(id); //No usar persistencia en domain

```

Regla: Los Servicios de Dominio deben ser puros

Utilidad: Deben ser fácilmente testeables, sin efectos colaterales ni dependencias externas.

Ejemplo:

```

public class CalculadoraDeDescuento {
    public BigDecimal calcular(BigDecimal total, int porcentaje) {
        return
        total.multiply(BigDecimal.valueOf(porcentaje)).divide(BigDecimal.valueOf(100));
    }
}

```

Regla: Las entidades no deben delegar responsabilidad al dominio.service

Utilidad: Las entidades deben mantener sus propias invariantes. El dominio.service solo debe utilizar entidades que ya estén validadas o en estado adecuado para aplicar una regla cruzada.

Regla: Un domain.service puede participar en múltiples casos de uso

Utilidad: Al estar centrado en reglas transversales, puede ser reutilizado desde múltiples application.service.

Ejemplo:

Un servicio CalculadorDeMultasTotal puede ser invocado tanto al crear un pedido como al validar uno.

5.7. Servicios de Aplicación

Regla: Implementar los casos de uso del negocio en los servicios de aplicación

Utilidad: Orquestan entidades, repositorios y lógica externa.

Ejemplo:

```
public PersonaResponse obtenerPersona(Long id) {
    Persona persona = repositorio.buscarPorId(id).orElseThrow(...);
    return mapper.toResponse(persona);
}
```

Regla: Los casos de uso coordinan, no implementan lógica de negocio

Utilidad: Un caso de uso (Application Service) orquesta el flujo de ejecución: recibe una entrada (DTO), delega la lógica al dominio, llama al repositorio para persistencia y devuelve un DTO como resultado.

No debe contener reglas de negocio propias ni acceder directamente a la base de datos.

Ejemplo:

```
public class GestionarPersonaService {

    public PersonaResponse obtenerPersona(Long id) {
        Persona persona = personaRepository.buscarPorId(id)
            .orElseThrow(() -> new NotFoundException("No existe persona"));

        return mapper.toResponse(persona);
    }
}
```

Regla: El caso de uso convierte DTOs ↔ Entidades

Utilidad: Separa la representación externa del modelo interno. El Application Service se apoya en mappers para transformar la entrada (DTO) en un objeto del dominio, y viceversa para la salida.

Ejemplo:

```
Persona persona = mapper.toDomain(request);
```

Regla: El Caso de Uso invoca servicios del dominio y repositorios

Utilidad: Aplica el patrón de coordinador. No tiene lógica propia, sino que delega a entidades del dominio o servicios de dominio para aplicar reglas, y a la capa de infraestructura para obtener o guardar datos.

Ejemplo:

```
persona.validarNui(); // lógica de dominio
personaRepository.guardar(persona); // persistencia delegada
```

Regla: No debe gestionar directamente errores de negocio ni validaciones complejas

Utilidad: Las validaciones y reglas de negocio deben estar encapsuladas dentro de entidades o servicios del dominio. Esto garantiza cohesión, reusabilidad y testabilidad de la lógica.

Ejemplo:

```
Correcto:
persona.validarNui(); // lógica delegada
Incorrecto:
if (persona.getEdad < 0) throw new IllegalStateException(...);
```

Regla: No accede directamente a la base de datos ni gestiona entidades JPA

Utilidad: Se trabaja siempre a través del repositorio del dominio, nunca con JPA o Panache

directamente. Esto evita el acoplamiento de la lógica de aplicación a detalles de persistencia.

Ejemplo:

```
Correcto:
personaRepository.buscarPorId(id);
Incorrecto:
PersonaEntity.findById(id); // rompe separación de capas
```

Regla: El Application Service es responsable de usar el Mapper

Utilidad: El Application Service es la única capa que conoce tanto los DTOs como el modelo de dominio. Es el lugar adecuado para realizar la transformación.

Ejemplo:

```
public PersonaResponse crearPersona(PersonaRequest request) {
    Persona persona= mapper.toDomain(request);
    repositorio.guardar(persona);
    return mapper.toResponse(persona);
}
```

5.8. Mappers (MapStruct)

Regla: Usar mappers para convertir DTOs ↔ Entidades

Utilidad: Centraliza la conversión entre representaciones externas (DTO) y el dominio. Mejora la separación de responsabilidades y evita lógica repetida en controladores o servicios.

Ejemplo:

```
@Mapper(componentModel = "cdi")
public interface PersonaMapper {
    Persona toDomain(PersonaRequest request);
    PersonaResponse toResponse(Persona persona);
}
```

Regla: Los DTO deben ser fácilmente serializables (JSON/XML)

Utilidad: Permiten interoperabilidad con clientes y sistemas externos. Su estructura debe ser simple, compatible con serializadores como JSON-B o Jackson.

Ejemplo: (para JSON-B en Quarkus):

```
public class PersonaResponse {
    public String codigo;
    public String nombre;

    public PersonaResponse() {} // Constructor vacío requerido
    public String getCodigo() { return codigo; }
    public void setCodigo(String codigo) { this.codigo = codigo; }
}
```

Regla: Usar para Mappers (MapStruct) para transformar entidades ↔ dominio ↔ DTOs.

Utilidad: Transforman DTO ↔ Dominio ↔ Entity.

Ejemplo:

```
@Mapper(componentModel = "cdi")
public interface PersonaMapper {
    Persona toDomain(PersonaEntity entity);
}
```

Regla: Usa @Mapper(componentModel = "cdi") en Quarkus

Utilidad: Garantiza integración con el sistema de inyección de dependencias de Quarkus (@Inject).

Ejemplo:

```
@Mapper(componentModel = "cdi") //Recomendado en Quarkus
```

Regla: Separar completamente los DTOs del dominio

Utilidad: Asegura bajo acoplamiento entre la capa de exposición (REST) y el núcleo de dominio. Evita exponer estructuras internas y mejora la mantenibilidad.

Ejemplo:

```

Persona persona = mapper.toDomain(request); // En Application
PersonaResponse response = mapper.toResponse(persona); // En Application

```

Regla: Usar MapStruct para conversiones entre entidades y DTOs

Utilidad: Genera automáticamente código de mapeo eficiente y limpio. Se evita la lógica manual repetitiva y propensa a errores.

Ejemplo:

```

@Mapper(componentModel = "cdi")
public interface PersonaMapper {
    Persona toDomain(PersonaRequest request);
    PersonaResponse toResponse(Persona persona);
}

```

Regla: Usa default solo para transformaciones simples

Utilidad: El método default en interfaces MapStruct permite escribir lógica personalizada mínima. No se recomienda agregar lógica pesada o validaciones complejas aquí.

Ejemplo:

```

default Codigo map(String valor) {
    return new Codigo(valor);
}

```

Regla: Centraliza los mappers por módulo o agregado

Utilidad: Facilita el mantenimiento y evita la duplicación de lógica de conversión. Cada agregado debe tener su propio Mapper.

Ejemplo:

```

PersonaMapper.java
DireccionMapper.java

```

Regla: No usar el Mapper directamente desde el REST Controller

Utilidad: Mantiene la separación de capas. El REST Controller no debe conocer el dominio ni las reglas de transformación.

Ejemplo:

```

Correcto:
return gestionarPersonaService.obtenerPersona(id);
Incorrecto:
return personaMapper.toResponse(domain);

```

Regla: Usar Mapper para convertir entre entidades de persistencia y objetos de dominio

Utilidad: Separa claramente los modelos de persistencia (con anotaciones JPA) de los modelos del dominio (puros POJOs con lógica de negocio).

Ejemplo:

```

Persona persona= mapper.toDomain(entity);
PersonaEntity entity = mapper.toEntity(persona);

```

5.9. Uso de DTOs (Request/Response)

Regla: Usar DTOs para la entrada y salida de los datos (Request/Response).

Utilidad: Transportan datos entre capas. No contienen lógica.

Ejemplo:

```

public class PersonaRequest {
    private String codigo;
}

```

Regla: Los DTO deben representar estructuras planas para la comunicación externa

Utilidad: Los DTO (Data Transfer Object) son usados para comunicar el microservicio con capas externas (REST, UI, mensajería), y no deben acoplarse al modelo de dominio. Esto permite una evolución independiente del dominio y evita exponer detalles internos.

Ejemplo:

```
public class PersonaRequest {  
    public String codigo;  
    public String nombre;  
}
```

Regla: No reutilizar entidades del dominio como DTO

Utilidad: Evita acoplamiento entre la API pública y el núcleo del dominio. Las entidades contienen lógica que no debe ser visible fuera del sistema.

Ejemplo:

```
Correcto:  
return mapper.toResponse(persona);  
Incorrecto:  
return persona; // devuelve entidad directamente
```

Regla: Usar REST en los Controllers para exponer los servicios

Utilidad: Solo reciben/retornan DTOs. No contienen lógica de negocio.

Ejemplo:

```
@GET @Path("/{id}")  
public PersonaResponse get(@PathParam("id") Long id) {  
    return service.obtenerPersona(id);  
}
```

Regla: El Controller representa la entrada al microservicio

Utilidad: Es la capa que interactúa directamente con el exterior (cliente, UI, otro servicio). Su rol es exponer los casos de uso definidos en la capa de aplicación mediante una API REST.

Ejemplo:

```
@Path("/personas")  
@Produces(MediaType.APPLICATION_JSON)  
@Consumes(MediaType.APPLICATION_JSON)  
public class PersonaResource {  
    @Inject  
    GestionarPersonaService service;  
  
    @GET.  
    @Path("/{id}")  
    public PersonaResponse obtener(@PathParam("id") Long id) {  
        return service.obtenerPersona(id);  
    }  
}
```

5.10. REST

Regla: El Rest debe comunicarse exclusivamente con la capa application

Utilidad: El Rest no debe conocer ni utilizar clases del dominio directamente (entidades, repositorios, VO, etc.), lo que garantiza el aislamiento de capas y bajo acoplamiento.

Ejemplo:

```
Correcto:  
PersonaResponse response = service.obtenerPersona(id);  
Incorrecto:  
persona.getCodigo(); // uso directo de dominio
```

Regla: El Rest recibe y retorna solo DTOs

Utilidad: Se evita filtrar internamente la estructura de dominio hacia los consumidores externos. Los DTOs son estructuras planas, fácilmente serializables.

Ejemplo:

```
@POST
public Response crear(PersonaRequest request) {
    PersonaResponse response = service.crearPersona(request);
    return Response.status(Response.Status.CREATED).entity(response).build();
}
```

Regla: La lógica de negocio NO debe estar en el Rest

Utilidad: Evita violaciones a la separación de responsabilidades. Toda la lógica debe ser delegada al caso de uso (application/service) correspondiente.

Ejemplo:

```
Correcto:
return service.crearPersona(request);
Incorrecto:
if (request.getPrecio().compareTo(BigDecimal.ZERO) < 0) {
    throw new RuntimeException("Precio inválido");
}
```

Regla: No usar el Mapper directamente en el Rest

Utilidad: El Mapper pertenece a la lógica de aplicación. El Rest debe delegar todo al caso de uso.

Ejemplo:

```
Correcto:
PersonaResponse response = service.obtenerPersona(id);
Incorrecto:
Persona persona= repositorio.findById(id);
PersonaResponse response = mapper.toResponse(persona);
```

5.11. Configuración de URI (Uniform Resource Identifier)

Regla: Los nombres de los URIs en las APIs REST deben seguir la convención de kebab-case³, donde las palabras están en minúsculas y separadas por guiones (-).

Utilidad: Mejora la legibilidad, la consistencia y la compatibilidad. El formato kebab-case hace que los URIs sean más fáciles de leer, especialmente en rutas largas.

Ejemplo:

Correcto (usando kebab-case)	Incorrecto (evitar usar camelCase o PascalCase)
/usuarios-activos /productos-destacados	/usuariosActivos /productosDestacados

Regla: El versionado de la API debe implementarse a través de la URL, utilizando números enteros para las versiones, precedidos por el prefijo "v" (por ejemplo, "v1", "v2"). Nunca se debe liberar una API sin asignarle un número de versión. Además, se deben evitar los números decimales en las versiones.

Utilidad: Permite realizar cambios en la API sin interrumpir a los servicios existentes, mejorando la evolución de esta sin causar problemas de compatibilidad.

Ejemplo:

Correcto			Incorrecto
Verbo	Versión 1	Versión 2	
GET	/api/v1/usuarios	/api/v2/usuarios	• /api/v1.1

³ Véase <https://opensource.zalando.com/restful-api-guidelines/#appendix-best-practices>

POST	/api/v1/usuarios	/api/v2/usuarios	• /api/v-1.1
PUT	/api/v1/usuarios/{id}	/api/v2/usuarios/{id}	
DELETE	/api/v1/usuarios/{id}	/api/v2/usuarios/{id}	

Regla: Las rutas (URIs) deben ser coherentes, legibles y seguir un esquema lógico basado en los recursos, evitando el uso de verbos en las mismas. Estas URIs deben adoptar una estructura jerárquica predecible, lo que mejora tanto su comprensibilidad como su usabilidad. El término "predecible" se refiere a que las rutas sean coherentes en su diseño, mientras que "jerárquico" indica que los datos están organizados de manera estructurada, reflejando las relaciones entre los recursos de forma clara y lógica.

Sintaxis:

`http://www.sri.gob.ec/{ bounded-context}/api/{version}/{recurso}/{id?}`

- **{bounded-context}**: El core de negocio o módulo general.
- **{version}**: La versión de la API.
- **{recurso}**: nombre del recurso.
- **{id?}**: Opcionalmente, un parámetro de identificación del recurso.

Utilidad: Mejora la legibilidad de las rutas y la mantenibilidad de la API mediante la consistencia en su diseño de la API, facilitando comprensión por parte de otros desarrolladores y usuarios.

Ejemplo:

Correcto	
GET	Obtener todos los sujetos. http://www.sri.gob.ec/sri-catastros/api/v1/sujetos
POST	Crear un nuevo sujeto. http://www.sri.gob.ec/sri-catastros/api/v1/sujetos
PUT	Actualizar un sujeto específico http://www.sri.gob.ec/sri-catastros/api/v1/sujetos/123
DELETE	Eliminar un sujeto específico http://www.sri.gob.ec/sri-catastros/api/v1/sujetos/123
GET	Obtener contactos de un sujeto http://www.sri.gob.ec/sri-catastros/api/v1/sujetos/123/contactos
Incorrecto	
GET	<code>http://www.sri.gob.ec/sri-catastros/api/v1/getSujetos</code>
DELETE	<code>http://www.sri.gob.ec/sri-catastros/api/v1/deleteSujeto/{id}</code>

Regla: Cuando no exista un concepto de colección o entidad de negocio es aceptable usar un nombre de recurso en singular.

```
/* Servicio que obtiene una configuración global desde el módulo de
Administración */
GET http://www.sri.gob.ec/sri-administracion-interfaz/rest/configuracion
```

Utilidad: Nombres de recursos apropiados proporcionan contexto para una solicitud de servicio, aumentando la comprensión de la API de servicio. Los recursos se ven jerárquicamente a través de sus nombres de URI, ofreciendo a los consumidores una jerarquía amigable y fácil de entender para aprovechar al máximo sus aplicaciones. Cuando los recursos se nombran bien, una API es intuitiva y fácil de usar.

Consideraciones:

- Las API REST están escritas para los consumidores. El nombre y la estructura de los URIs deben transmitir significado. Diseñe para sus clientes, no para sus datos.
- Los recursos que se ven jerárquicamente a través de los URIs es un análisis sobre la granularidad del servicio, contexto de negocio y esquema relacional.

Regla: Los errores deben ser respondidos de manera estructurada y consistente en formato JSON, con un mensaje claro y un código de estado HTTP adecuado. Para los cual debe cumplir las siguientes condiciones:

- El body de la respuesta DEBE devolver un objeto JSON que incluya una lista de errores con al menos un objeto de error.

Cada objeto de error en la lista DEBE incluir las siguientes claves:

- Código: identificador numérico único para la clase de error. Su objetivo es ayudar a solucionar problemas.
- Título: Descriptor único legible para la clase de error. Destinado a ayudar a la resolución de problemas.
- Detalle: mensaje legible para el usuario que describe el error. Destinado a que los sujetos lo muestren a los usuarios.

Los mensajes de error deben ser claros, descriptivos y gramaticalmente correctos, de manera que puedan ser presentados por los consumidores de la API sin necesidad de modificaciones.

- Cada mensaje de error DEBE empezar con mayúscula.
- Cada mensaje de error DEBE ser una oración completa.
- Cada mensaje de error DEBE concluir con un punto (.).

Utilidad: Facilita la identificación y solución de problemas, y asegura que los desarrolladores reciban información consistente sobre las fallas en la API.

Ejemplo:

```
{
  "errores": [
    {
      "codigo": "SRI-010"
      "titulo": "Razón social"
      "detalle": "No puede contener valores numéricos."
    },
    {
      ...
    }
  ]
}
```

```

    }
  ]
}

```

Regla: Permitir que la API devuelva respuestas en varios formatos (JSON y XML), según lo que se especifique en el encabezado Accept.

Utilidad: Proporciona flexibilidad a los consumidores de la API, permitiendo que puedan elegir el formato que mejor se adapte para cubrir las necesidades y/o requerimientos.

Ejemplo:

- Si el sujeto envía Accept: application/json, la respuesta será en JSON.
`curl -X GET "http://www.sri.gob.ec/sri-catastros/api/v1/sujetos" \`
`-H "Accept: application/json"`

```

/**
 *Especifica que este endpoint produce JSON
 */

@GET
@Produces(MediaType.APPLICATION_JSON)
public Response obtenerDatos() {
    return new Temperatura("Madrid", 15, 25);
}

```

- Si el sujeto envía Accept: application/xml, la respuesta será en XML.
`curl -X GET "http://www.sri.gob.ec/sri-catastros/api/v1/sujetos" \`
`-H "Accept: application/xml"`

```

/**
 *Especifica que este endpoint produce XML
 */

@GET
@Produces(MediaType.APPLICATION_XML)
public Response obtenerDatos() {
    return new Temperatura("Madrid", 15, 25);
}

```

Regla: Las solicitudes GET:

- PUEDEN incluir parámetros de consulta.
- NO DEBEN incluir un body de solicitud.

Utilidad: Recupera un único recurso o una lista de recursos. DEBE ser idempotente.

Ejemplo:

Mostrar recurso individual
GET <code>http://www.sri.gob.ec/sri-catastros/api/v1/sujetos/132</code>
Lista de colección de recursos
GET <code>http://www.sri.gob.ec/sri-catastros/api/v1/sujetos?estado=ACT</code>

Regla: Las solicitudes POST:

- NO DEBEN incluir parámetros de consulta.
- PUEDEN incluir un body de solicitud

Utilidad: Se utiliza para crear un recurso, cargar un archivo o activar una acción.

Ejemplo:

Crear un recurso
POST <code>http://www.sri.gob.ec/sri-catastros/api/v1/sujetos</code>

```
{
  "nombre": "Juan",
  "apellido": "Perez"
}
```

Regla: Las solicitudes PUT:

- NO DEBEN incluir parámetros de consulta.
- DEBE incluir en el body la representación completa del recurso.
- El recurso debe ser sustituido completamente o no cambiar en absoluto.

Utilidad: Se utiliza para actualizar todo un recurso.

Ejemplo:

```
Actualizar un recurso completo
PUT http://www.sri.gob.ec/sri-catastros/api/v1/sujetos/123
{
  "id": 123
  "nombre": "Juan",
  "apellido": "Gomez"
}
```

Regla: Las solicitudes PATCH:

- NO DEBEN incluir parámetros de consulta.
- PUEDEN incluir un cuerpo de solicitud.
- DEBEN aplicar todas las actualizaciones solicitadas o ninguna.

Utilidad: Se utiliza para actualizar una parte del recurso con los datos del body en la solicitud.

Ejemplo:

```
Actualizar parcialmente un recurso
PATCH http://www.sri.gob.ec/sri-catastros/api/v1/sujetos/123
{
  "apellido": "Castro"
}
```

Regla: Las solicitudes DELETE:

- PUEDEN incluir parámetros de consulta.
- NO DEBEN incluir un cuerpo de solicitud.
- Eliminar un recurso TAMBIÉN PUEDE eliminar recursivamente recursos asociados.
- La eliminación de un recurso PUEDE ocurrir de forma sincrónica o asincrónica.

Utilidad: Se utiliza para eliminar un recurso.

Ejemplo:

```
Eliminar el recurso
DELETE http://www.sri.gob.ec/sri-catastros/api/v1/sujetos/:id
```

5.12. Respuestas

Regla: Los servicios RESTful PUEDEN retornar `jakarta.ws.rs.core.Response`. Deben retornarlo cuando se requiera controlar explícitamente códigos HTTP, headers, Location, cookies o metadatos. En casos simples, PUEDEN retornar directamente el DTO (Quarkus serializa automáticamente).

Utilidad: El objeto `jakarta.ws.rs.core.Response` permite agregar metadatos como cabeceras y códigos de respuesta.

Ejemplo:

```
@GET
@Path("/{messageId}")
@Produces(MediaType.APPLICATION_JSON)
public Response getMessage(@PathParam("messageId") long id) {
    Map<String, String> map = new HashMap<>();
    map.put("1", "abc");
    map.put("2", "def");
    map.put("3", "ghi");
    return Response.status(Status.OK).entity(map).build();
}
Output: { "1": "abc", "2": "def", "3": "ghi" }
```

6. Seguridad

La seguridad es un principio transversal en la construcción de servicios backend institucionales. No solo garantiza la confidencialidad, integridad y disponibilidad de la información, sino que además asegura el cumplimiento de normativas nacionales (LOPD) e internacionales (ISO 27001, OWASP Top 10, NIST).

En el contexto de este estándar, la seguridad se implementa de forma “por defecto” (security by design), integrando prácticas de parametrización, autenticación centralizada, separación de ambientes, control de accesos y uso de canales cifrados. Adicionalmente, se establecen lineamientos específicos para el manejo de datos privados, autorización basada en roles y protección frente a vulnerabilidades comunes en APIs.

Regla: En el código fuente de las aplicaciones NO se deben fijar los valores de: direcciones IP, nombres de dominio, URLs, puertos, protocolos, nombre de servidor, correo electrónico, usuarios, claves o cualquier otro valor que sea identificado como potencialmente variable en el tiempo de vida del sistema, todos estos valores deben ser manejados mediante parametrización.

Utilidad: Facilita la portabilidad de la aplicación y evita exposición de secretos que comprometan la seguridad.

Regla: Todas las aplicaciones deben estar integradas al Single Sign On institucional.

Utilidad: Centraliza la autenticación, reduce riesgos de gestión de credenciales y facilita el cumplimiento normativo.

Regla: Los componentes transaccionales que requieran acceder a la base de datos, deben realizar las conexiones según su ambiente (Internet e Intranet). Es decir, si el componente corresponde a funcionalidad que se ejecuta en la intranet solo podrá conectarse a las bases de datos de ese ambiente (ejemplo: INTRASRI, SRI) y si el componente es de internet solo podrá conectarse a las bases de internet (ejemplo: INTER, INTERSRI). Para casos de excepción a esta regla se deberá contar por cada componente con la aprobación del Arquitecto de Tecnologías asignado al proyecto y la revisión del Departamento de Seguridad Informática.

Utilidad: Manejar el bajo acoplamiento en las aplicaciones y la independencia entre ambientes.

Regla: En la invocación desde un cliente (navegador) a cualquier URL web tales como servicios web (RESTful / SOAP), páginas web desarrolladas en cualquier tecnología, servlets u otra lógica a nivel del servidor, no se deben pasar como parámetros cualquier información que se pueda obtener mediante otro mecanismo, como por ejemplo de las cabeceras de la petición http, de información de sesión y/o token o que pudiera ser consultada en los repositorios institucionales en base a los datos anteriores.

Ejemplo:

No se debe enviar como parámetro los datos de:

- IPs: en su lugar se debe obtener del request.
- Número de identificación, nombre o correo del usuario: en su lugar se debe consultar en la base de datos con la identificación de la sesión.

Utilidad: Reducir riesgos de seguridad como, por ejemplo, que un contribuyente pueda obtener información de otro.

Regla: En el caso de que se requiera un servicio web para obtener información que constituyen datos privados (que le pertenece al exclusivamente usuario autenticado), se debe validar que las consultas retornen únicamente datos de ese usuario.

Utilidad: Reducir riesgos de seguridad como, por ejemplo, que un contribuyente pueda obtener información de otro.

Ejemplo: Cuando un usuario ingresa al sistema y consulta una de sus declaraciones, se debe asegurar que dicha consulta no solo se realice la búsqueda en base al código de la declaración, sino que también filtre por la identificación de la persona que está autenticada en sesión.

Regla: La autorización de los endpoints debe estar basado en roles de usuario (autorización basada en roles), los cuales deberán ser parametrizables usando un aplicativo para este propósito.

Utilidad: Garantiza que el acceso a los recursos esté controlado según el perfil del usuario, evitando accesos indebidos. La parametrización centralizada permite ajustar permisos sin necesidad de cambios en el código, reduciendo riesgos y costos de mantenimiento.

Regla: El consumo de los servicios debe realizarse a través del protocolo seguro HTTPS.

Utilidad: Asegura la confidencialidad e integridad de los datos transmitidos, previniendo ataques de interceptación (man-in-the-middle) y garantizando la confianza en las comunicaciones entre clientes y servidores

Regla: El consumo de los servicios debe realizarse únicamente mediante el protocolo **HTTPS (TLS 1.2 o superior, preferible TLS 1.3)**.

Utilidad: Garantiza la confidencialidad e integridad de los datos en tránsito.

Regla: Los tokens de acceso (JWT u OAuth2) deben tener tiempos de expiración limitados y ser firmados digitalmente.

Utilidad: Reduce riesgos de robo de tokens y garantiza su validez.

7. Eventos

Los eventos permiten ejecutar acciones específicas ante situaciones relevantes en el ciclo de vida de una aplicación backend, tales como el inicio o apagado de la misma. En el marco de Quarkus y Jakarta

EE, el uso de eventos facilita la inicialización controlada de recursos, la liberación ordenada de conexiones y la ejecución de tareas programadas o de monitoreo.

Este capítulo define los lineamientos para el uso institucional de eventos, limitando su alcance a los casos estrictamente necesarios y bajo un control normativo que garantice consistencia y resiliencia.

Regla: Los únicos eventos autorizados para uso institucional son:

- `io.quarkus.runtime.StartupEvent`
- `io.quarkus.runtime.ShutdownEvent`

Utilidad: Estos eventos permiten gestionar de manera estandarizada la inicialización y finalización de recursos, evitando comportamientos inesperados en entornos productivos.

Regla: Para el uso de eventos adicionales no contemplados en este estándar, se debe solicitar la revisión y aprobación del **Arquitecto de Soluciones Tecnológicas al proyecto**.

Utilidad: Asegura control sobre el crecimiento de complejidad y evita dependencias innecesarias en el ciclo de vida de la aplicación.

Regla: La lógica implementada en los listeners de eventos debe ser **ligera y no bloqueante** (ejemplo: inicialización de recursos, configuración de logging, carga de parámetros).

Utilidad: Garantiza que el tiempo de arranque y apagado de la aplicación sea predecible, evitando cuellos de botella o fallos por tiempos de espera.

Regla: Los recursos inicializados durante un `StartupEvent` deben ser liberados explícitamente en un `ShutdownEvent`.

Utilidad: Previene fugas de memoria, conexiones huérfanas y problemas de estabilidad en ambientes de alta concurrencia.

8. Java Persistence API – JPA

El acceso a datos es un componente crítico en los sistemas institucionales, y su correcta implementación asegura la integridad, rendimiento y mantenibilidad de las aplicaciones. El uso de JPA **con Hibernate Reactive Panache** permite abstraer la lógica de persistencia, facilitando un desarrollo más ágil, orientado a entidades y alineado con las buenas prácticas del framework Quarkus.

Este capítulo establece las reglas para el uso institucional de JPA, abarcando aspectos de paginación, gestión de conexiones, procedimientos almacenados, entidades, clases embebibles y mapeo de relaciones, con el objetivo de garantizar consistencia técnica y prevenir malas prácticas en la capa de persistencia.

Regla: Para el acceso a datos se debe utilizar el ORM Hibernate Reactive With Panache provisto por la extensión `io.quarkus:quarkus-hibernate-reactive-panache`, esto para: mapear estructuras de base de datos, ejecutar cualquier sentencia DML (Data Manipulation Language) o ejecución de algún procedimiento almacenado o paquete. Solo se debe utilizar JDBC en sentencias DML, procedimientos o paquetes que tengan que ver con gran cantidad de datos y las mismas estén dando problemas de rendimiento; el Arquitecto de Soluciones Tecnológicas asignado debe validar la factibilidad de uso de este mecanismo y garantizar la reutilización de componentes genéricos.

Utilidad: Estandariza la forma de interactuar con la base de datos usando un ORM reactivo moderno, optimizando la escalabilidad y reduciendo el código repetitivo. El uso controlado de JDBC evita cuellos de botella y asegura que cualquier excepción al estándar sea revisada técnicamente para mantener consistencia y eficiencia.

Regla: No se debe utilizar directamente APIs específicas del proveedor JPA (por ejemplo, `org.hibernate.*`) en las capas de dominio y aplicación. El acceso a persistencia debe realizarse mediante JPA estándar (`jakarta.persistence.*`) y/o repositorios definidos por el dominio. El uso de APIs específicas del proveedor se permite únicamente en la capa de infraestructura, de forma encapsulada, con justificación técnica y aprobación del Arquitecto de Soluciones Tecnológicas asignado.

Utilidad: Evita acoplamientos a implementaciones propietarias, garantizando portabilidad y mantenibilidad del código. Esto permite migrar de versión o proveedor de JPA sin impactar el núcleo del sistema, reduciendo riesgos a futuro.

Regla: Para consultas que devuelven múltiples registros en **Hibernate Reactive**, se deben retornar tipos **reactivos** (`Uni<List<T>>` / `Multi<T>`) usando Panache Reactive o consultas reactivas (Mutiny).

Utilidad: Promueve la separación de responsabilidades al mantener las consultas en el DAO y las validaciones en la capa de servicios, reduciendo acoplamiento y mejorando la claridad del código. Además, estandariza el acceso a resultados múltiples, facilitando la mantenibilidad y la legibilidad.

8.1. Pool de conexiones

Regla: El tamaño del pool debe configurarse usando las propiedades del **datasource reactivo**. La parametrización se define con Arquitectura TI.

Utilidad: Establece un uso eficiente de recursos y evita saturación de la base de datos.

Regla: No se permite el uso de consultas nativas o `NativeQuery` salvo casos excepcionales autorizados por el Arquitecto de Soluciones Tecnológicas

Utilidad: Mantiene portabilidad entre proveedores de base de datos y mejora la mantenibilidad del código.

Regla: Si se requiere utilizar en las consultas los operadores como `LIKE`, `OR`, operadores de agregación, y junturas de más de cuatro tablas, que reducen el rendimiento, es necesario validar previamente con el Arquitecto de Soluciones asignado para determinar su viabilidad o alternativas de uso.

Utilidad: Previene degradación de rendimiento y permite evaluar alternativas más eficientes.

8.2. Procedimientos almacenados

Regla: La invocación de procedimientos almacenados desde JPA, requiere la revisión y aprobación del Arquitecto de Soluciones Tecnológicas asignado.

Utilidad: Evita la inclusión de lógica de negocio compleja en la base de datos y promueve un diseño desacoplado.

Regla: Para los escenarios aprobados de ejecución de procedimientos almacenados estos no deben contener lógica compleja y su tiempo de procesamiento debe ser menor o igual a 3 minutos.

Utilidad: Garantiza que los procedimientos no afecten la disponibilidad ni generen bloqueos prolongados en la base de datos.

8.3. Entidades

Regla: Las anotaciones JPA deben ser puestas sobre los atributos los mismos que hacen referencia a los campos de la tabla mapeada.

Utilidad: Mejora la claridad y evita inconsistencias en el mapeo.

Regla: Cada entidad debe tener id o clave primaria (primary key) y para denotar a un atributo o campo como clave primaria se debe usar la anotación `@Id` (paquete: `jakarta.persistence.Id`). El nombre del atributo de la clave primaria debe ser '`codigo`' y tipo de dato '`Long`'.

Utilidad: Uniformiza el modelo de datos y simplifica la trazabilidad.

Regla: No se permite el uso de claves primarias compuestas (primary key), a excepción de mapeos a aplicaciones legadas donde se tengan claves primarias compuestas.

Utilidad: Favorece la simplicidad y mantenibilidad del modelo de datos, al usar llaves primarias simples y estables. Esto mejora el rendimiento en consultas, facilita integraciones con ORM como Hibernate/Panache y evita complejidades en relaciones, índices y migraciones. Las excepciones para aplicaciones legadas permiten compatibilidad sin comprometer la consistencia general de la arquitectura.

8.4. Enumeraciones de dominio

Regla: Uso de enum en el dominio

Utilidad: Un enum encapsula un conjunto cerrado y constante de valores predefinidos, ideales para representar categorías o estados del dominio que no deben modificarse en tiempo de ejecución.

Ejemplo:

```
public enum Estado {  
    ACTIVO,  
    INACTIVO,  
    SUSPENDIDO  
}
```

Regla: Los enum DEBEN representar valores finitos, bien definidos por el dominio **Utilidad:** Se usan cuando los estados posibles son inmutables, esto mejora la claridad, reduce errores y evita el uso de cadenas mágicas o números sin significado.

Ejemplo:

```
public enum Prioridad { BAJA, MEDIA, ALTA }"
```

Regla: Los enum PUEDEN contener lógica adicional si mejora la expresividad del dominio

Utilidad: Si la lógica está relacionada directamente con el estado representado por el enum, se puede incluir comportamiento (métodos) dentro del mismo enum.

Ejemplo:

```
public enum Estado {  
    ACTIVO,  
    INACTIVO;  
    public boolean estaActivo() {  
        return this == ACTIVO;  
    }  
}
```

Regla: NO colocar lógica de negocio compleja dentro de un enum

Utilidad: Los enum deben mantener su simplicidad. Si la lógica es extensa o involucra múltiples entidades o decisiones, se debe mover a una clase de servicio o entidad del dominio.

Regla: Los enum del dominio NO deben tener anotaciones JPA

Utilidad: Para mantener la pureza del modelo de dominio, los enum no deben depender de JPA ni

tener anotaciones como `@Enumerated`. Su mapeo debe manejarse en la capa de infraestructura (por ejemplo, en `@Entity` con `@Enumerated`).

Regla: Usar enum en lugar de String o int para representar categorías fijas

Utilidad: Mejora la legibilidad del código, previene errores por valores inválidos y facilita el mantenimiento a largo plazo.

Ejemplo:

```
Correcto:
private Estado estado; // claro, validado, seguro
Incorrecto:
private String estado; // propenso a errores y difícil de validar
```

8.5. Repositorios de Infraestructura

Regla: La implementación del repositorio debe estar en `infraestructura.persistence.repository`

Utilidad: La clase concreta que utiliza Panache o JPA vive en infraestructura y es la encargada de convertir entre entidades JPA ↔ entidades del dominio mediante `MapStruct`.

Ejemplo:

```
@ApplicationScoped
public class PersonaRepositoryImpl implements PersonaRepository {
    @Inject
    PersonaMapper mapper;

    public Persona buscarPorId(Long id) {
        var entity = PersonaEntity.findById(id);
        return mapper.toDomain(entity);
    }
}
```

8.6. Clases embeddable en entidades

Regla: Se debe utilizar clases 'embeddables' (insertables) cuando:

- Más de una entidad utilice esa clase 'embeddable'.
- Por ampliación del dominio de la entidad se requiere agregar nuevos atributos donde por granularidad fina en el modelado del diseño se puede utilizar una clase 'embeddable' para estos nuevos atributos.
- El diseño relacional no se ajusta al modelado orientado a objetos (por ejemplo, una vista que involucra varias tablas se puede representar en varios objetos).

Utilidad: Promueve la reutilización y reduce duplicación de código.

Regla: No se deben utilizar clases 'embeddables' para solucionar problemas de PMD (regla `TooManyFields`) en entidades.

Utilidad: Asegura que las clases embebibles respondan a un diseño lógico y no a un bypass de validaciones.

Regla: Los Value Objects que representan estado de una entidad y no poseen identidad propia deben declararse con `@jakarta.persistence.Embeddable` y embeberse en una entidad con `@jakarta.persistence.Embedded` (o `@EmbeddedId` cuando corresponda). El tipo de acceso `@Access(AccessType.FIELD|PROPERTY)` no se impone de forma general; por consistencia y simplicidad se recomienda `@Access(AccessType.FIELD)`. El uso de `@Access(AccessType.PROPERTY)` se permite solo como excepción justificada (por ejemplo, cuando

se requiere validación/transformación obligatoria en getters/setters o encapsulación estricta).

Notas normativas:

- Se debe utilizar `jakarta.persistence.*` (no `javax.persistence.*`).
- Si el mismo `@Embeddable` se reutiliza en múltiples entidades y cambian los nombres de columna, se deben aplicar `@AttributeOverride` / `@AttributeOverrides` en cada entidad.

Utilidad: Permite modelar atributos compuestos reutilizables (Value Objects) como parte del estado de una entidad sin duplicar columnas ni lógica, manteniendo un diseño cohesivo y evitando acoplamiento innecesario. Al embeberse en la entidad propietaria, las columnas quedan administradas dentro de la misma tabla, garantizando persistencia consistente y mayor mantenibilidad.

8.7. Mapeo de relaciones en entidades

Regla: Para modelar relaciones entre entidades JPA se deben utilizar obligatoriamente las anotaciones (decoradores) de relación según la cardinalidad del modelo:

- **Uno a uno:** `@OneToOne`
- **Uno a muchos:** `@OneToMany`
- **Muchos a uno:** `@ManyToOne`
- **Muchos a muchos:** `@ManyToMany` (solo si no requiere atributos adicionales; caso contrario usar entidad intermedia con `@ManyToOne`)
- En relaciones con clave foránea, se debe declarar `@JoinColumn` en el **lado propietario**.
- En relaciones bidireccionales, el lado inverso debe declarar `mappedBy`.

Utilidad: Estandariza el mapeo relacional orientado a objetos, evita diseños ambiguos (FKs duplicadas, tablas puente inesperadas), reduce errores de persistencia y mejora la mantenibilidad y consistencia del modelo.

8.8. Repositorios de Dominio

Regla: Solo deben declarar métodos necesarios para ejecutar la lógica del dominio

Utilidad: Se evita incluir métodos genéricos innecesarios como `findAll()` o `deleteAll()` si no aportan valor al modelo de dominio. Los métodos deben estar alineados con los requerimientos del negocio.

Ejemplo:

```
Correcto:
List<Persona> buscarPorCategoria(String categoriaId);
Incorrecto:
List< Persona> findAll();
```

Regla: El contrato de los repositorios (interfaces) debe aceptar y retornar únicamente Entidades de Dominio, Value Objects o enums. La implementación en Infrastructure puede usar entidades JPA y frameworks, pero debe mapear `Entity` ↔ `Domain` y nunca filtrar tipos de Infrastructure hacia el core.

Utilidad: Evita el acoplamiento del dominio/aplicación a la tecnología de persistencia, mejora testabilidad, facilita migraciones (JPA ↔ otro), y protege el modelo ante cambios de infraestructura.

Regla: El repositorio del dominio no debe tener anotaciones, ni lógica técnica

Utilidad: Mantiene la pureza del modelo. Las anotaciones como `@Repository`, `@Transactional`, `@Inject` o el uso de `Panache` deben limitarse a la capa de infraestructura.

Ejemplo:

```

Correcto:
public interface PersonaRepository {
    Persona guardar(Persona persona);
}
Incorrecto:
@Repository
public interface PersonaRepositoryJpa extends JpaRepository< PersonaEntity, Long> { ... }

```

8.9. Entidades de Persistencia en Infraestructura

Regla: Usar PanacheEntity o PanacheRepository en la infraestructura, nunca en el dominio

Utilidad: Las entidades Panache son específicas de la persistencia (JPA), y no deben contaminar la capa de dominio. La lógica de acceso se limita a la infraestructura.

Ejemplo:

```
PersonaEntity.find("estado", Estado.ACTIVO).list();
```

Regla: Las entidades de persistencia solo se usan en la capa de infraestructura

Utilidad: Asegura un bajo acoplamiento. No deben salir de esta capa ni exponerse en controladores, servicios de aplicación o dominio.

Ejemplo:

```

Correcto:
Persona persona = mapper.toDomain(entity);
Incorrecto:
public PersonaEntity obtenerPersona() // usado en application.service

```

Regla: Evita lógica de negocio en las entidades de persistencia

Utilidad: La entidad JPA/Panache solo representa una estructura de almacenamiento. La lógica de negocio debe estar encapsulada en las entidades del dominio.

Ejemplo:

```

Correcto:
Esa lógica debe ir en domain.model.entidad.Persona
Incorrecto:
public boolean estaActivo() { return this.estado.equals("ACTIVO"); }

```

Regla: Utilizar PanacheEntity o PanacheEntityBase según necesidades

Utilidad: PanacheEntity: simplifica la definición del ID (Long id ya incluido).

PanacheEntityBase: permite definir tu propio identificador con mayor control.

Ejemplo:

```

public class PersonaEntity extends PanacheEntity { }
// o
public class PersonaEntity extends PanacheEntityBase {
    @Id Long personaId;
}

```

Regla: Usa Panache Reactive solo si tu aplicación es verdaderamente reactiva

Utilidad: Panache Reactive es útil para aplicaciones de alta concurrencia, I/O intensivo o que usen reactive programming (Vert.x, Mutiny).

Cuando NO usarlo:

Si tu aplicación es síncrona (bloqueante).

Si no estás usando programación reactiva en todo el stack.

Ejemplo:

```
public Uni<PersonaEntity> findByCodigo(String codigo);
```

Regla: Las entidades deben tener constructores vacíos, y opcionalmente getters/setters

Utilidad: Necesario para el funcionamiento de frameworks de serialización (Hibernate, JSON-B, Jackson).

Ejemplo:

```
public PersonaEntity() {} // Constructor vacío requerido
- **persistence.mapper**: Mappers MapStruct
```

Regla: Implementación concreta de los repositorios en la capa de infraestructura

Utilidad: Implementan interfaces del dominio. Devuelven objetos del dominio.

Ejemplo:

```
@ApplicationScoped
public class PersonaRepositoryImpl implements PersonaRepository {
    public Optional<Persona> buscarPorId(id) { ... }
}
```

Regla: Implementar interfaces de repositorio definidas en el dominio

Utilidad: Mantiene la inversión de dependencias: el dominio define qué se necesita (la interfaz), y la infraestructura define cómo se accede (la implementación).

Esto permite desacoplar el dominio de detalles de persistencia.

Ejemplo:

```
public class PersonaRepositoryImpl implements PersonaRepository {
    @Override
    public Persona buscarPorId(Long id) {
        PersonaEntity entity = PersonaEntity.findById(id);
        return mapper.toDomain(entity);
    }
}
```

Regla: La implementación del repositorio debe estar anotada con `@ApplicationScoped`

Utilidad: Permite que la clase sea administrada por el contenedor CDI de Quarkus y se pueda inyectar en los servicios de aplicación.

Ejemplo:

```
@ApplicationScoped
public class PersonaRepositoryImpl implements PersonaRepository { ... }
```

Regla: El repositorio de infraestructura puede incluir búsquedas especializadas (consultas JPQL, criterios, etc.)

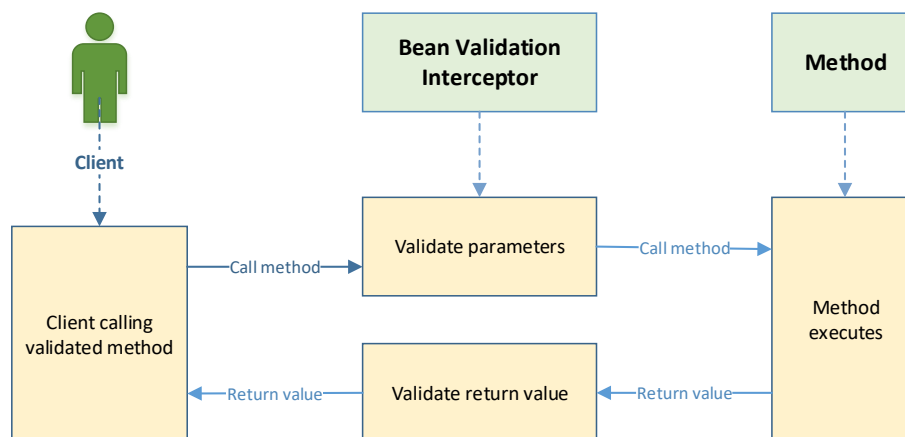
Utilidad: Siempre que el resultado se convierta a un objeto del dominio, puede implementar queries personalizadas con filtros complejos.

Ejemplo:

```
public List<Persona> buscarPorNombre(String nombre) {
    return PersonaEntity.find("nombre like ?1", "%" + nombre + "%")
        .list()
        .stream()
        .map(mapper::toDomain)
        .toList();
}
```

9. Validaciones de los Java Beans

El API Java para Validación de JavaBeans (Bean Validation) proporciona un mecanismo para la validación de datos de la aplicación. Bean Validation define un modelo de **constraints** (restricciones) basado en anotaciones, las mismas que pueden ser colocadas sobre un campo(field), método o una clase de un componente JavaBeans, como un managed bean o una Entity. La siguiente imagen muestra el ciclo de vida de Bean Validation al ejecutar métodos.



Ciclo de vida de Bean Validation al ejecutar métodos.

9.1. Constraints de Bean Validation

Regla: Todos los campos o propiedades que tengan los constraints (`@Size`, `@Digits`, `@DecimalMin`, `@DecimalMax`, `@Valid`) debe ir precedido por el constraint `@NotNull`.

Ejemplos:

```

@NotNull(message = "{invalid.firstName}")
@Size(min=2, max=10, message = "{invalid.sizeName}")
private String firstName;

@NotNull(message = "{invalid.notNullTotalSalary}")
@Digits(integer=4, fraction=2, message = "{invalid.totalSalaryNumberRange}")
private BigDecimal totalSueldo;

@POST
@Path("/ingresar")
@Produces(MediaType.APPLICATION_JSON)
public String ingresar(@NotNull(message = "{contact.object.notNull}") @Valid Contacto contacto){
    return "EXITO";
}
  
```

Importante:

Si dentro de una entidad de persistencia se pone en algún campo (field) o atributo validaciones de la anotación `jakarta.persistence.Column` conjuntamente con un constraint de Bean Validation, la validación que tiene prioridad en ejecutarse es la de Bean Validation. Esto debido a que la validación de `@Column` que es de JPA se ejecuta en la capa de persistencia, mientras que los constraint de Bean Validation se ejecutan en la capa de presentación.

9.2. Personalización de mensajes de error de validación

Regla: Se debe personalizar y estandarizar los mensajes de error de validación usando códigos de mensaje en las anotaciones de Bean Validation y un archivo `ValidationMessages.properties` (y sus variantes por idioma si aplica). Queda prohibido escribir mensajes “mensajes literales fijados directamente en el código” directamente en las anotaciones. Los mensajes deben ser consistentes, reutilizables y trazables (código + texto).

Utilidad: Garantiza respuestas de error uniformes en toda la API, facilita mantenimiento y cambios masivos (sin tocar código), permite internacionalización y mejora la experiencia de consumo (front/otros servicios) al tener mensajes previsibles.

Ejemplo:

DTO con códigos de mensaje

```
public class CrearUsuarioRequest {  
  
    @NotBlank(message = "{usuario.username.requerido}")  
    @Size(min = 3, max = 60, message = "{usuario.username.tamano}")  
    public String username;  
  
    @NotBlank(message = "{usuario.email.requerido}")  
    @Email(message = "{usuario.email.formato}")  
    public String email;  
}
```

Archivo de mensajes

src/main/resources/ValidationMessages.properties

```
usuario.username.requerido=El nombre de usuario es obligatorio.  
usuario.username.tamano=El nombre de usuario debe tener entre 3 y 60 caracteres.  
usuario.email.requerido=El correo es obligatorio.  
usuario.email.formato=El correo no tiene un formato válido.
```

9.3. Validación de servicios web RESTFul

Regla: Los parámetros y cuerpos de solicitud de los endpoints RESTFul (JAX-RS) que requieran restricciones deben validarse mediante Bean Validation (Jakarta Validation), utilizando constraints (por ejemplo `@NotNull`, `@Size`, `@Pattern`) y mensajes personalizados definidos en el archivo `ValidationMessages.properties` ubicado en `src/main/resources`.

Utilidad: Permite aplicar un esquema de validación estandarizado y reutilizable en las APIs RESTFul, garantizando consistencia en la verificación de entradas y facilitando la gestión centralizada de mensajes de error.

Ejemplo:

```
import jakarta.validation.constraints.NotNull;  
import jakarta.ws.rs.GET;  
import jakarta.ws.rs.Path;  
import jakarta.ws.rs.QueryParam;  
import jakarta.ws.rs.core.Response;  
  
@Path("/sujetos")  
public class SujetoResource {  
  
    /**  
     * Obtiene todos los sujetos del módulo de catastro por estado  
     */  
    @GET  
    public Response obtenerPorEstado(  
        @NotNull(message = "{sujeto.consulta.estado.requerido}")  
        @QueryParam("estado") String estado  
    ) {  
        return Response.ok().build();  
    }  
}
```

10. Componentes JEE

10.1. Inyección de Dependencias y Contextos (CDI)

Regla: La inyección de dependencias se lo debe hacer con la anotación `@Inject`.

Utilidad: Establece un mecanismo estándar, desacoplando las clases de su construcción y facilitando la mantenibilidad, el testing y la integración con el contenedor CDI.

Regla: Para definir el *scope* se debe usar por defecto la anotación `@ApplicationScoped`. Si se requiere el uso de otro tipo de *scope* debe solicitarse la asignación de un Arquitecto de Soluciones para realizar una revisión conjunta.

Utilidad: Garantiza un ciclo de vida controlado y consistente para los componentes. El uso de `@ApplicationScoped` evita instanciaciones innecesarias y asegura eficiencia en recursos; la revisión de excepciones previene malas prácticas o fugas de memoria.

Regla: Para hacer la inyección de dependencia de los beans (POJOs) implementados con o sin cualificadores, se debe inyectar usando la anotación `@Inject` (paquete `jakarta.inject.Inject`).

Utilidad: CDI da la facilidad de inyectar POJOs con solo utilizar la anotación `@Inject` y cumplir la especificación Jakarta EE.

Ejemplo: Ver el proyecto base en [Anexo 1: Recursos Relacionados](#).

10.1.1. Scopes

Regla: En lo que respecta las anotaciones de los estados (scopes) de los beans se debe cumplir lo siguiente:

- Para los beans que necesiten instanciarse una sola vez a nivel de aplicación

Anotación	Descripción
<code>@ApplicationScoped</code>	Se utiliza para definir un bean como un bean con ámbito de aplicación, lo que significa que se creará solo una vez por aplicación y se compartirá entre todos los clientes. Esto puede resultar útil para beans cuya creación es costosa o para beans que deben compartirse entre varios clientes.
<code>@Singleton</code>	A diferencia de <code>@ApplicationScoped</code> , si un bean tiene la anotación <code>@Singleton</code> , se crea una instancia cuando se inyecta en otro bean. Pero es irrelevante si alguna vez se accede al bean.

- Para los beans que necesitan instanciarse por cada solicitud.

Anotación	Descripción
<code>@RequestScoped</code>	Si una clase de bean tiene la anotación <code>@RequestScoped</code> , CDI creará una instancia del bean de forma perezosa durante la primera llamada a un método de bean. Un bean de este tipo vive sólo dentro de una cadena que se utiliza para procesar una única solicitud HTTP.

10.1.2. Interceptores

Regla: Se debe analizar con el Arquitecto de Soluciones Tecnológicas asignado la viabilidad de utilizar interceptores en el desarrollo de la aplicación.

Utilidad: Permite evaluar el impacto y pertinencia del uso de interceptores, asegurando que se apliquen únicamente cuando aporten valor real (como auditoría, logging o seguridad transversal) y evitando un uso excesivo que complique el mantenimiento o afecte el rendimiento. La revisión por parte del Arquitecto de Soluciones Tecnológicas garantiza consistencia con las buenas prácticas institucionales y con la arquitectura definida.

10.1.3. Decoradores

Regla: Se debe analizar con el Arquitecto de Soluciones Tecnológicas asignado la viabilidad de utilizar decoradores en el desarrollo de la aplicación, debido a que los decoradores pueden conducir a un sistema a la existencia de una gran cantidad de decoradores similares que pueden ser difíciles de mantener y distinguir.

Utilidad: Asegura que el uso de decoradores se limite a escenarios justificados, evitando la proliferación innecesaria de implementaciones que compliquen el mantenimiento, reduzcan la claridad del diseño y generen ambigüedad en el comportamiento del sistema. La validación por parte del Arquitecto de Soluciones Tecnológicas garantiza un balance adecuado entre flexibilidad y simplicidad en la arquitectura de la aplicación.

11. Manejo de logs y excepciones

11.1. Manejo de Logs / pistas de auditoría

Regla: Para habilitar observabilidad desacoplada, los logs o pistas de auditoría de los microservicios deben capturarse con Fluent Bit, publicarse en un tópico dedicado de Kafka y ser consumidos por Logstash para su indexación en Elasticsearch (visualización en Kibana); el flujo debe poder activarse/desactivarse (escalando o eliminando los DaemonSet/Deployment) para evitar consumo innecesario de recursos.

Utilidad: Kafka desacopla la generación de logs/ pistas de auditoría del almacenamiento/búsqueda, permitiendo reintentos y amortiguar picos; Elasticsearch centraliza e indexa para consulta rápida en Kibana; y la capacidad de detener Fluent Bit/Logstash controla costos y evita llenar memoria/almacenamiento cuando no se esté realizando la prueba o monitoreo.

Regla: Los tópicos de Kafka deben nombrarse con una estructura jerárquica que identifique de la siguiente manera:

`sri.<solucion>.<bounded-context>.<capacidad>.<tipo-mensaje>.<entorno>`

Se deben usar únicamente minúsculas y separadores con punto (.), evitando nombres ambiguos o abreviaciones no controladas.

Utilidad:

Permite identificar rápidamente el origen y propósito del tópico, facilita la gobernanza, el control de accesos y la retención de datos, y habilita el enrutamiento automático por patrones en consumidores como Logstash, manteniendo consistencia entre equipos y entornos.

Ejemplos:

`sri.comprobantes.recepcion.log.prod`
`sri.comprobantes.validacion.auditoria.prod`

Regla: Los logs o pistas de auditoría de aplicativos en Kubernetes deben enviarse a Elasticsearch usando Fluent Bit (DaemonSet), capturando únicamente stdout/stderr del aplicativo requerido (filtrado por namespace/app) y publicando en índices diarios.

Utilidad: Centraliza la trazabilidad y facilita diagnóstico en Kibana, reduce ingesta innecesaria al filtrar solo el aplicativo, y evita consumo de recursos por reintentos cuando hay credenciales inválidas.

Regla: Las pruebas unitarias no deben implementar logs.

Utilidad: Mantiene las pruebas unitarias simples, enfocadas en validar el comportamiento de la lógica sin depender de salidas externas. Esto facilita su ejecución automatizada, reduce el ruido en los resultados y promueve la repetibilidad.

11.1.1. Implementación de mensajes de logs

Regla: Todos los mensajes logs deben poseer un código de proyecto general de máximo 9 caracteres según el contexto de negocio y un identificador de rango según su nivel.

Utilidad: Estandariza la identificación de logs, permitiendo clasificarlos por proyecto y nivel de severidad. Esto mejora la capacidad de búsqueda, el monitoreo centralizado y la correlación de eventos en herramientas de observabilidad.

Nivel	Identificador	Ejemplo
DEBUG	100	101, 102, ... 199
INFO	200	201, 202, ... 299
WARN	300	301, 302, ... 399
ERROR	400	401, 402, ... 499
FATAL	500	501, 502, ... 599

Regla: A continuación, se definen las reglas para usar niveles de log:

- **DEBUG:** Usar este nivel siempre que se esté investigando un error específico.
- **INFO:** Usar este nivel para emitir información general como: inicio, apagado, versión y configuración de una aplicación. También para emitir información acerca de decisiones importantes en el código o cambios en la configuración.
- **WARN:** Usar este nivel para emitir una advertencia o información que no representan un problema por sí mismo.
- **ERROR:** Usar este nivel para identificar un problema y se tienen que tomar acción sobre esto.
- **FATAL:** Usar este nivel si la aplicación presenta un problema con un estado irreparable como por ejemplo excepciones lanzadas por la JVM (OutOfMemoryError), cuando esté relacionado a un apagado del servidor o a matar la sesión.

Utilidad: Establece un uso uniforme de los niveles de log, evitando ambigüedad en su aplicación. Esto permite interpretar correctamente la severidad de los eventos, facilita la gestión operativa y mejora el monitoreo con herramientas de observabilidad.

Regla: El nivel del log que se utiliza en producción es: "ERROR".

Utilidad: Reduce el volumen de logs en ambientes críticos, registrando únicamente incidentes relevantes que requieren atención, lo cual optimiza almacenamiento, rendimiento y análisis en producción.

Regla: El nivel del log que utilizan en los ambientes no productivos (Testing y Certificación) es: "DEBUG".

Utilidad: Permite mayor nivel de detalle para depuración y validación durante las fases de desarrollo y pruebas, facilitando la identificación y resolución temprana de defectos antes del paso a producción.

Regla: La gestión de errores debe centralizarse usando excepciones controladas

Utilidad: Permite una gestión uniforme de errores y códigos HTTP mediante ExceptionMappers en Quarkus o filtros globales.

Ejemplo:

```
throw new NotFoundException("Persona no encontrado"); // Propagado al mapper global
```

Regla: shared.exception debe contener solo excepciones personalizadas reutilizables

Utilidad: Centraliza y normaliza el manejo de errores. Permite lanzar excepciones semánticas desde cualquier capa de la aplicación, evitando usar excepciones genéricas como RuntimeException.

Ejemplo:

```
public class NotFoundException extends RuntimeException {
    public NotFoundException(String message) {
        super(message);
    }
}

public class ValidationException extends RuntimeException {
    public ValidationException(String message) {
        super(message);
    }
}
```

Buenas prácticas:

Nombres claros que indiquen el tipo de error.

No capturar ni lanzar directamente desde el controlador sin pasarlo por la capa de aplicación.

Usar una jerarquía si se requiere: DomainException, ApplicationException, etc.

11.2. Manejo de Excepciones

11.2.1. Implementación de excepciones

Regla: Toda excepción controlada que ocurra debe capturarse en el método donde se produce y transformarse a una excepción de aplicación (ej. BusinessException, NotFoundException, ConflictException) con un código funcional. No se deben propagar excepciones técnicas (SQL, NullPointerException, etc.) como si fueran de negocio.

Utilidad: Clasifica correctamente el error, mantiene el contexto del caso de uso, mejora trazabilidad y evita que detalles internos se filtren a capas superiores o terminen expuestos en la API.

12. Lógica parametrizable

Regla: Para el uso de la librería de reglas de negocio (Drools) se requerirá contar con el análisis y aprobación del Arquitecto de Soluciones Tecnológicas asignado y se deberá colocar la definición correspondiente en la Arquitectura de la Solución.

Regla: En el caso del uso de la librería de reglas de negocio (Drools), se debe almacenar el archivo de

reglas de negocio en la base de datos.

12.1. Utilitarios Compartidos

Regla: shared.utils debe contener solo funciones auxiliares reutilizables

Utilidad: Encapsula lógica común que no pertenece al dominio, pero es necesaria en múltiples lugares (e.g. validaciones simples, formatos, conversiones, encriptación).

Reglas de uso:

Evitar lógica de negocio aquí (eso debe ir en el dominio).

Evitar abusar del uso de utils; cada clase debe tener responsabilidad bien definida.

Opcional: agrupar por funcionalidad (por ejemplo: DateUtils, FileUtils, SecurityUtils).

Ejemplo:

```
public class StringUtils {
    public static boolean isNullOrEmpty(String value) {
        return value == null || value.trim().isEmpty();
    }
}
```

Regla: Nunca referenciar entidades del dominio ni infraestructuras desde shared

Utilidad: Evita dependencias cruzadas y mantiene shared como una capa puramente técnica y utilitaria, accesible por todas las demás capas sin generar acoplamiento.

Ejemplo:

Correcto:

StringUtils, DateValidator, NotFoundException.

Incorrecto:

PersonaValidator con lógica de negocio (esto va en domain.service).

13. Glosario

Término / Acrónimo	Definición (para glosario)
API (Application Programming Interface)	Interfaz que expone capacidades o recursos para ser consumidos por otros sistemas o clientes.
REST	Estilo arquitectónico que define principios para diseñar sistemas distribuidos basados en recursos, comunicación sin estado y una interfaz uniforme sobre HTTP.
RESTful	Término que describe a los servicios o APIs que implementan correctamente los principios del estilo arquitectónico REST.
HTTP (Hypertext Transfer Protocol)	Protocolo de comunicación; define métodos (GET/POST/PUT/DELETE, etc.) y códigos de estado.
URI (Uniform Resource Identifier)	Identificador uniforme de un recurso expuesto por una API (ruta del recurso).
JSON (JavaScript Object Notation)	Formato estándar de intercambio de datos usado en requests/responses.
OpenAPI	Especificación para diseñar y documentar APIs REST (contrato de endpoints, modelos, respuestas).
JAX-RS	Especificación Java (Jakarta) para construir servicios REST.
Jakarta EE	Plataforma empresarial Java sobre la cual se estandarizan componentes backend.
Quarkus	Framework Java para microservicios (optimizado para contenedores y

Término / Acrónimo	Definición (para glosario)
	tiempos de arranque).
JWT (JSON Web Token)	Token para autenticación/autorización, normalmente enviado como Bearer en el header Authorization.
OAuth2	Estándar de autorización para proteger APIs mediante tokens y flujos de acceso.
CDI (Contexts and Dependency Injection)	Especificación para inyección de dependencias y manejo de contextos (scopes) en aplicaciones Java.
DDD (Domain-Driven Design)	Enfoque de diseño centrado en el dominio del negocio, separando responsabilidades por capas y contextos.
Bounded Context	Límite lógico del dominio (DDD) donde un modelo y su lenguaje ubicuo son consistentes.
Dominio (Domain)	Capa donde residen entidades, value objects y reglas del negocio, evitando acoplamientos técnicos.
VO (Value Object)	Objeto (generalmente inmutable) que representa un concepto del dominio por su valor, no por identidad.
DTO (Data Transfer Object)	Objeto para transportar datos entre capas o entre cliente-servidor; sin lógica de negocio.
Capa de Aplicación (Application)	Capa que orquesta casos de uso: coordina dominio e infraestructura sin contener detalles técnicos.
Infraestructura (Infrastructure)	Capa técnica: controladores, persistencia, integración, mapeos y adaptadores a frameworks.
JPA (Java Persistence API)	Estándar de persistencia ORM para mapear entidades y gestionar acceso a base de datos.
Embeddable	Tipo JPA embebible reutilizable dentro de una entidad (componente persistente sin identidad propia).
Panache	Enfoque/librería de Quarkus para simplificar repositorios y acceso a datos.
MapStruct	Herramienta de mapeo (compile-time) para transformar entre entidades, DTOs y modelos de dominio.
TDD (Test Driven Development)	Técnica donde se escriben pruebas antes del código para guiar el diseño y reducir defectos.
JUnit	Framework de pruebas unitarias para Java.
Mockito / Mocks	Framework y técnica para simular dependencias y aislar unidades bajo prueba.
CI/CD	Automatización de integración, validaciones de calidad y despliegue mediante pipelines.
PMD	Análisis estático para detectar “code smells”, malas prácticas y reglas de codificación.
Checkstyle	Herramienta para validar convenciones de estilo y formato de código.
SpotBugs	Análisis estático para detección de defectos comunes y riesgos (nulls, concurrencia, etc.).
OWASP API Security Top	Referencia de los riesgos más comunes de seguridad en APIs y controles

Término / Acrónimo	Definición (para glosario)
10	recomendados.
Vault	Solución para gestión segura de secretos (credenciales, llaves, tokens) fuera del código.
ConfigMap	Recurso de Kubernetes para parametrización no sensible (configuración).
Secret	Recurso de Kubernetes para parametrización sensible (datos codificados/gestionados como secretos).

14. Bibliografía

1. Quarkus (2023), Guides - Latest: Tutorials Short and focused exercises to get you going quickly, <https://quarkus.io/guides/>.
2. Eclipse EE4j (2023), The Jakarta EE Tutorial, <https://eclipse-ee4j.github.io/jakartaee-tutorial/>
3. RESTful API Design. (s.f.). URL Structure. Recuperado de <https://restful-api-design.readthedocs.io/en/latest/urls.html>
4. API Guide. (s.f.). Discoverability. Recuperado de <https://apiguide.readthedocs.io/en/latest/principles/discoverability.html>
5. JSON:API. (s.f.). JSON:API specification. Recuperado de [https://jsonapi.org/OpenAPI Specification. \(s.f.\). Encoding the x-www-form-urlencoded media type. Recuperado de https://spec.openapis.org/oas/latest.html#encoding-the-x-www-form-urlencoded-media-type](https://jsonapi.org/OpenAPI Specification. (s.f.). Encoding the x-www-form-urlencoded media type. Recuperado de https://spec.openapis.org/oas/latest.html#encoding-the-x-www-form-urlencoded-media-type)
6. Fielding, R., & Reschke, J. (2005). RFC 3986: Uniform Resource Identifier (URI): Generic Syntax (Sección 1.1.2). Recuperado de <https://datatracker.ietf.org/doc/html/rfc3986#section-1.1.2>
7. Auth0. (n.d.). Rate limit use cases. Auth0. <https://auth0.com/docs/troubleshoot/customer-support/operational-policies/rate-limit-policy/rate-limit-use-cases#api-responses>
8. Swagger. (n.d.). About the OpenAPI Specification. Swagger. https://swagger.io/docs/specification/v3_0/about/
9. Mozilla Developer Network. (s.f.). Códigos de estado HTTP. MDN Web Docs. Recuperado de <https://developer.mozilla.org/es/docs/Web/HTTP/Status>
10. Fielding, R., & Reschke, J. (2014). Hypertext Transfer Protocol (HTTP/1.1): Semantics and Content (RFC 7231). Internet Engineering Task Force (IETF). Recuperado de <https://datatracker.ietf.org/doc/html/rfc7231#section-6.5.1>

15. Anexos

15.1. Anexo 1: Recursos Relacionados

Recurso	Ubicación
Archivo de reglas PMD	sri-reglas-pmd.md · main · Arquetipos / reglas-codigoestatico-java / sri-reglas-pmd · GitLab
Archivo de reglas CheckStyle	Arquetipos / reglas-codigoestatico-java / sri-reglas-checkstyle · GitLab
Reglas SpotBugs	sri-reglas-spotbugs.md · main · Arquetipos / reglas-codigoestatico-java / sri-reglas-spotbugs · GitLab

Proyecto Base	Arquetipos / piloto-base-microservicios / sri-base-api · GitLab Arquetipos / piloto-base-microservicios / sri-administracion-catalogo-api · GitLab https://gitlab.sri.ad/Arquetipos/piloto-base-microservicios/sri-autorizacion-api
Contexts And Dependency Injection Guide	https://es.quarkus.io/guides/cdi-reference

15.2. Anexo 2: Estándares Relacionados

Estándar	Descripción	Ubicación
Interacción con la base de datos	Cumplir los lineamientos establecidos para cada base de datos que existen en la institución, estos lineamientos son de cumplimiento obligatorio y se encuentran descritos en el documento Estándar de Desarrollo – Nombrado de Objetos de Base de Datos , si existiera un caso de excepción a esta norma, debe ser resuelta y acordada con el Departamento de Planificación TI y Gestión de Información -Datos y el Departamento de Infraestructura y Operaciones, estas definiciones deben ser reflejadas en el documento de Arquitectura de la Solución Tecnológica de cada aplicación	https://sriintra01/share/page/document-details?nodeRef=workspace://SpacesStore/3bf7bcf9-1b96-43b2-b06e-7e64d0346edd
Estándar Tecnológico de Seguridad de Aplicaciones	Definir las mejores prácticas en materia de seguridad de aplicaciones	https://sriintra01/share/page/document-details?nodeRef=workspace://SpacesStore/45ea4978-85d2-4076-bbc4-f9917cf79072
Estándar Tecnológico de Seguridad de Aplicaciones Java	Definir las mejores prácticas en materia de seguridad en el desarrollo de aplicaciones bajo plataforma Java	https://sriintra01/share/page/document-details?nodeRef=workspace://SpacesStore/c268bc0c-6e1d-4398-bd69-538627653a9b
Estándar Tecnológico de Seguridad de API Web	Estandarizar los lineamientos técnicos para la aplicación de seguridad de API Webs institucionales o que interactúan con la institución	https://sriintra01/share/page/document-details?nodeRef=workspace://SpacesStore/5280cfa-8a1b-4d46-bad6-7dc9bf90d0b6
Estándar Tecnológico de Protección de Datos Personales	El documento tiene como objetivo proporcionar reglas para la protección de datos personales en las etapas del ciclo de vida del desarrollo de software de soluciones tecnológicas institucionales y el manejo de sus ambientes	https://sriintra01/share/page/document-details?nodeRef=workspace://SpacesStore/2adc5cfd-ccb7-45ad-b357-9485cf858122

15.3. Anexo 3: Versiones de tecnologías a utilizar

Tecnología	Versión
Java Development Kit (JDK)	Red Hat – OpenJDK 21.x
Java Enterprise Edition	Jakarta EE 10

Apache Maven	>=3.0.0
Framework para el desarrollo de microservicios	>= Quarkus 3.9.25
Implementación de JAX-RS	Quarkus extension: io.quarkus:quarkus-resteasy-reactive
OpenAPI	>=3.0.0
GraalVM	GraalVM Community Edition 21.x (compatible con Red Hat – OpenJDK 21.x)

15.4. Anexo 4: Estructura (scaffolding)

Estructura - Domain	Detalle
domain.model.entity	Contiene las entidades del dominio. Estas representan conceptos con identidad y comportamiento. No deben tener anotaciones técnicas como @Entity.
domain.model.vo	Define los Value Objects (VO), que son inmutables, representan conceptos del dominio y encapsulan reglas de validación simples.
domain.model.enums	Incluye enumeraciones que representan estados o categorías fijas del dominio.
domain.service	Servicios de dominio que encapsulan reglas de negocio transversales que no pertenecen exclusivamente a una entidad.
domain.repository	Interfaces que definen contratos de persistencia requeridos por el dominio, sin implementar detalles técnicos.
Estructura - infrastructure	Detalle
application.service	Casos de uso que coordinan la lógica de negocio, acceso a repositorios y transformación entre entidades y DTOs.
application.dto	DTOs usados para recibir o enviar datos en los endpoints. No contienen lógica de negocio.
Estructura - application	Detalle
infrastructure.persistence.entity	Entidades JPA para persistencia. Reflejan la estructura de base de datos, separadas del dominio.
infrastructure.persistence.mapper	Mappers de infraestructura que transforman entre entidades JPA y entidades del dominio usando MapStruct.
infrastructure.persistence	Contiene las implementaciones de los repositorios definidos en el dominio, usando Panache.
infrastructure.rest	Controllers REST que exponen endpoints y usan casos de uso desde la capa de

	aplicación.
Estructura - shared	Detalle
shared.exception	Define excepciones personalizadas reutilizables en todas las capas.
shared.response	Modelos de respuesta común para respuestas estándar de la API.
shared.utils	Clases utilitarias de uso transversal, sin acoplamiento a lógica de negocio.

Flujo para construir un microservicio con arquitectura DDD, Quarkus, MapStruct y Panache de acuerdo con la estructura base.

	Detalle
Dominio (Domain)	Crear entidades en `domain.model.entidad`
	Crear objetos de valor (VO) en `domain.model.vo`
	Crear enumeraciones en `domain.model.enums`
	Crear servicios de dominio (opcional) en `domain.service`
	Definir interfaces de repositorio en `domain.repository`
Aplicación (Application)	Crear DTOs en `application.dto`
	Crear servicios de aplicación (casos de uso) en `application.service`
Infraestructura (Infrastructure)	Crear entidades JPA en `infrastructure.persistence.entity`
	Crear mappers con MapStruct en `infrastructure.persistence.mapper`
	Implementar interfaces de repositorio en `infrastructure.persistence.repository`
	Crear controladores REST en `infrastructure.rest`
Compartido (Shared)	Definir excepciones en `shared.exception`
	Crear estructuras de respuesta en `shared.response`
	Añadir utilitarios si se requieren en `shared.utils`
Pruebas	Crear pruebas unitarias e integración bajo `src/test/java`

Gráfico de relación de las capas

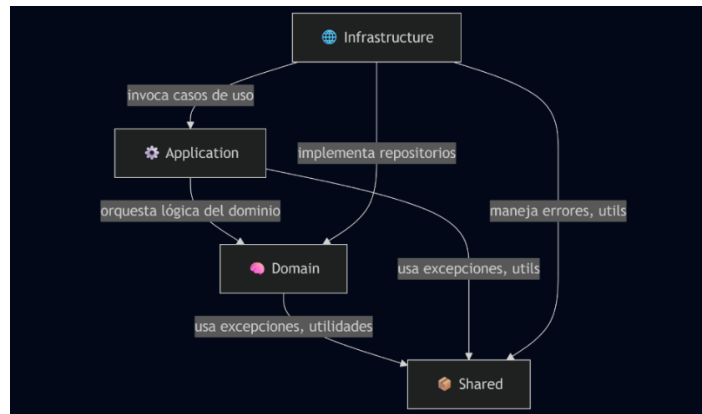


Gráfico de comunicación de los componentes de cada capa

